



**MGM's College of Engineering and Technology
Kamothe, Navi Mumbai-410209**

Department of Computer Engineering

Laboratory Manual

Skill Base Lab Course - Python Programming

Course Code: CSL405

Second Year

Semester- IV

Academic Year 2023-24

CONTENTS

S. No.	TOPIC	PAGE NO.
1	<i>Vision and Mission of College and Department</i>	3
2	<i>Teaching Scheme, Examination Scheme and University Syllabus</i>	4
3	<i>Lab Objectives and Lab Outcomes (COs)</i>	5
4	<i>Program Specific Outcomes (PSOs) & Lab Outcomes (LOs)</i>	6
5		
6		
7		
8		
9		
10		
11		
12		
13		
14		
15		
16		
	<i>and registration (also validating user detail using regular expression).</i>	
17	<i>Experiment No 10: Programs on Threading using python.</i>	62
18	<i>Experiment No 11 Exploring basics of NumPy Methods.</i>	67
19	<i>Experiment No 12: Program to send email and read content of URL.</i>	77

MGM's College of Engineering and Technology

Vision

To be a leading, innovative and vibrant institution.

Mission

To create a world of technology, competitive globally.

Vision

To make a competent citizen.

To prepare students to face tomorrow.

Mission

1. To provide a high quality education.
2. To create a better societal

3. To establish Center of Excellence in various domains of Computer Engineering and promote active research and development.
4. To enhance the competency of the faculty in the latest technology through continuous development programs.
5. To foster networking with alumni and industries.

Study and Evaluation Scheme

Course Code	Course Name	Teaching Scheme			Credits Assigned			
CSL405	Skill Base Lab	Theory	Practic	Tutorial	Theory	Practica	Tutorial	Total
								2

Cou	
C	Total
	25

1	
2	
3	
4	ormance of
5	marks)

Program Specific Outcomes (PSOs)

The Computer Engineering graduates will be able to

PSO 1	Acquire skills to design, analyses and develop algorithms and implement them using high-level programming languages
PSO 2	Contribute their engineering skills in computing and information engineering domains like <i>ring.</i>
PSO 3	<i>nting and</i> <i>life.</i>

Engi

PO1) *and d* *fundamentals,*

PO2) *engin* *complex*
scien *s, natural*

PO3) *system* *and design*
publi *on for the*

PO4) *metho* *research*
inform *sis of the*

PO5) *engin* *and modern*
unde *es with an*

PO6) *socio* *to assess*
profes *ant to the*

PO7) Environment and sustainability: *Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.*

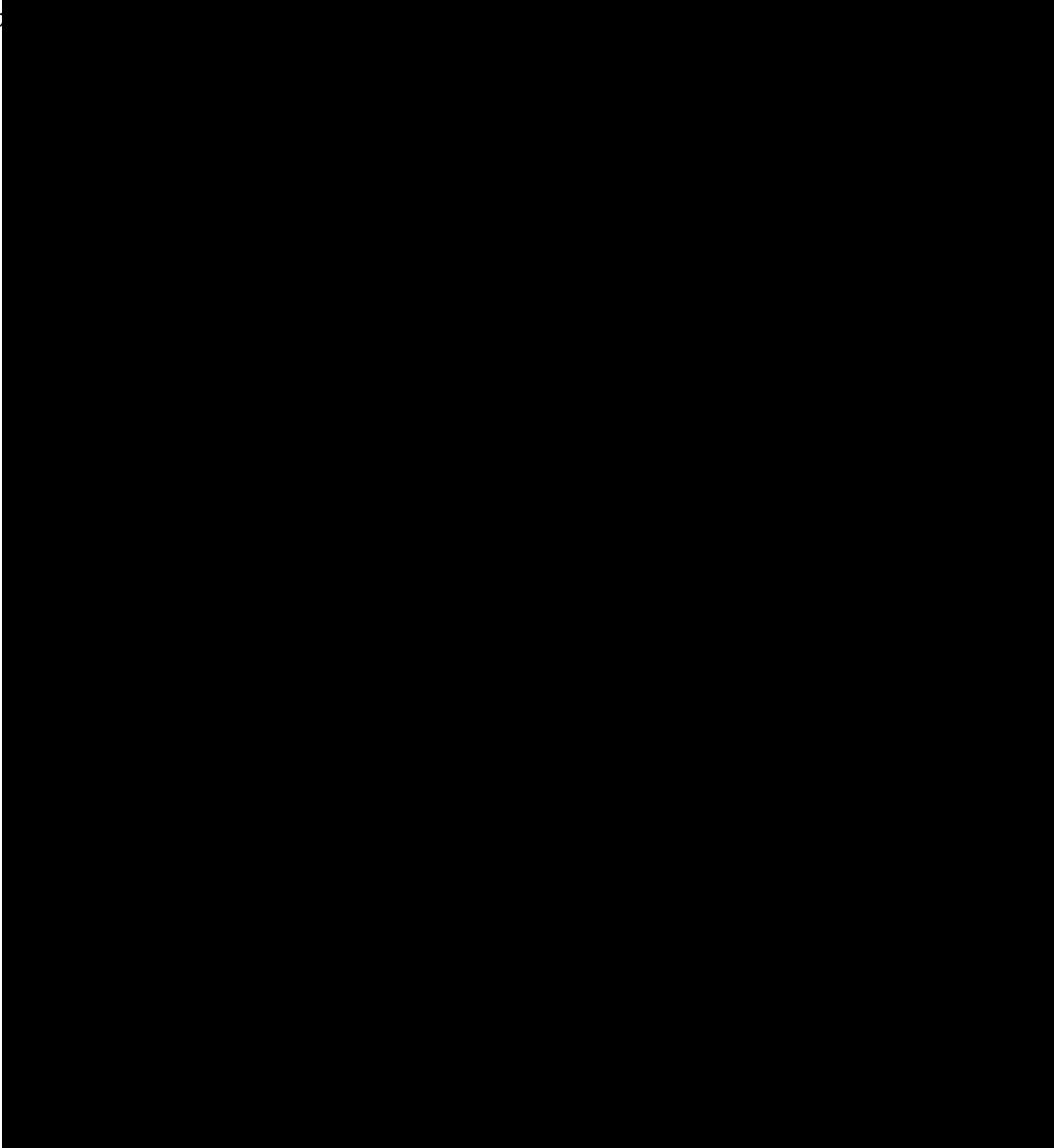
PO8) Ethics: *Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.*

PO9) Individual and team work: *Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.*

PO10) Communication: *Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.*

PO11) Project management and finance: *Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.*

PO12) Life-long learning: *Recognize the need for, and have the preparation and ability to engage in independent and lifelong learning in the broadest context of their own and others' intellectual and economic environments.*



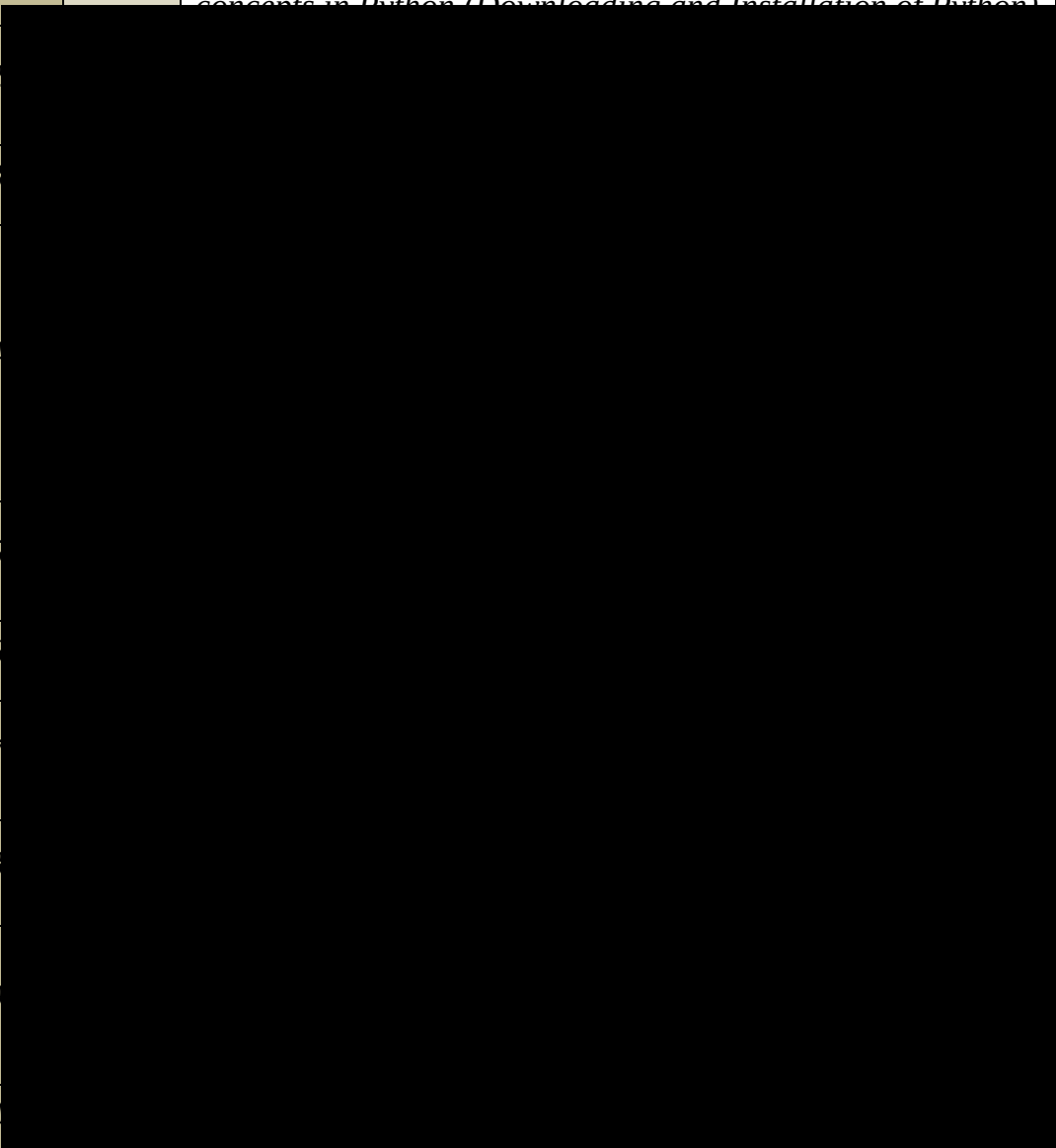
Mapping Lab Outcomes (CO) - Program Outcomes (PO)

Subject Weight	Course Outcomes	Contribution to Program outcomes												
		P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	P12	
PRATICAL										2	1	1	1	
										1	1	2	1	
										1	1	2	3	
										1	1	2	3	
										1	1	2	1	

List of Experiments

Sr. No.	Experiments Name
1	Experiment 01: Introduction to Python & basic programming concepts in Python. (Downloading and Installation of Python)
2	Experiment 02: Exploring basics of python like data types (strings,
	python.
	display
	acters
	ry
	ts such
	g built
	d,
	sing
	on
	tration
	URL.

Experiment Plan

Module No.	Week No.	Experiments Name	Course Outcome
1	W1	Experiment 01: Introduction to Python & basic programming concepts in Python (Downloading and Installation of Python)	
2			CO1
3			CO5
4			CO5
5			CO5
6			CO5
7			CO5
8			CO5
9			CO3
10			CO3
11	W11	Experiment No 11 Exploring basics of NumPy Methods.	CO4
12	W12	Experiment No 12: Program to send email and read content of URL.	CO4

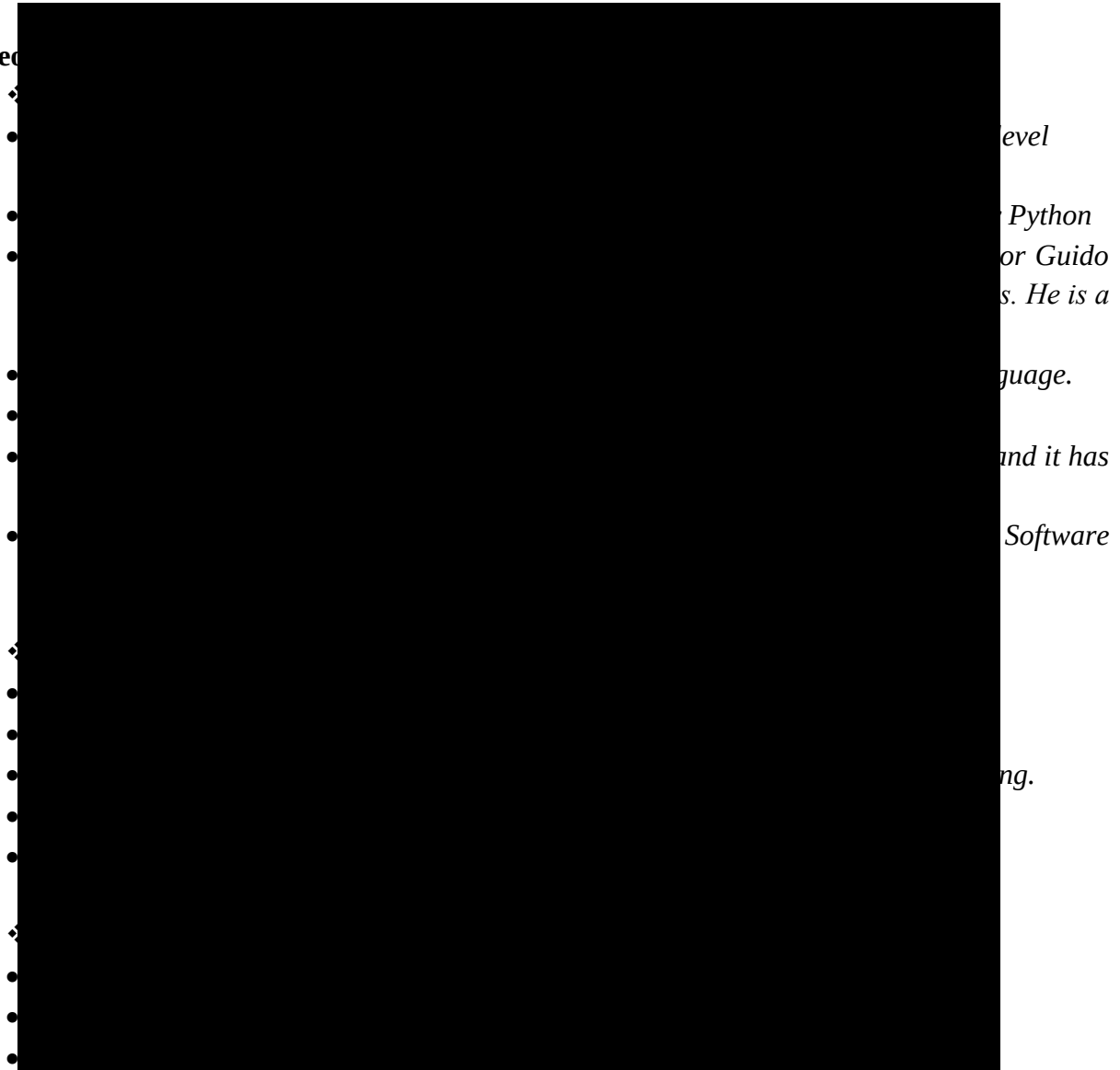
Experiment No. 1

Name of Student			
Roll No.			
DOP	DOS	Marks/Grade	Sign

Aim: Introduction to Python & basic programming concepts in Python.(Downloading and Installation of Python)

Software Required: Python3.9 / Online Editor

Theory



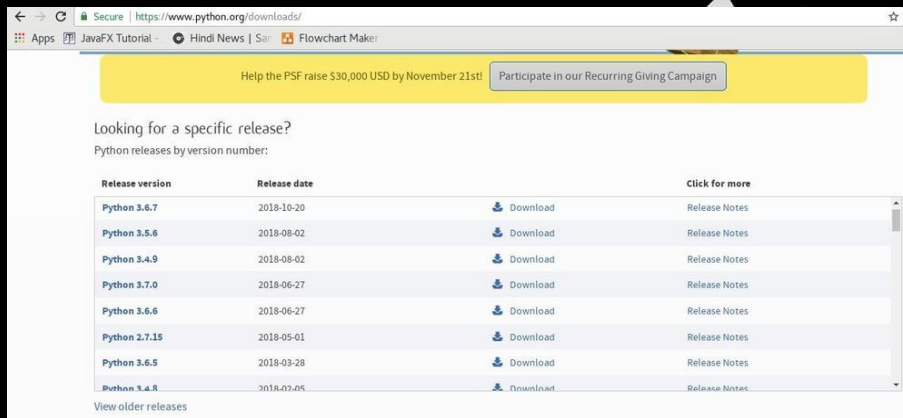
- Component Integration
- Database Programming
- Numeric and Scientific Programming

- *Gaming, Images, Serial Ports, XML, Robots, and More*

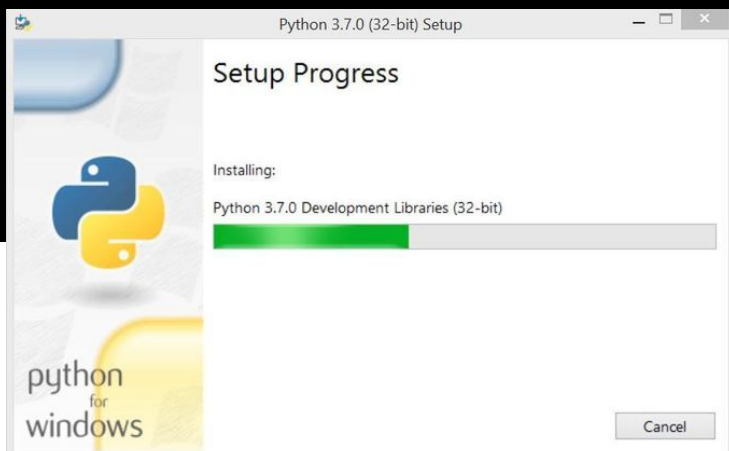
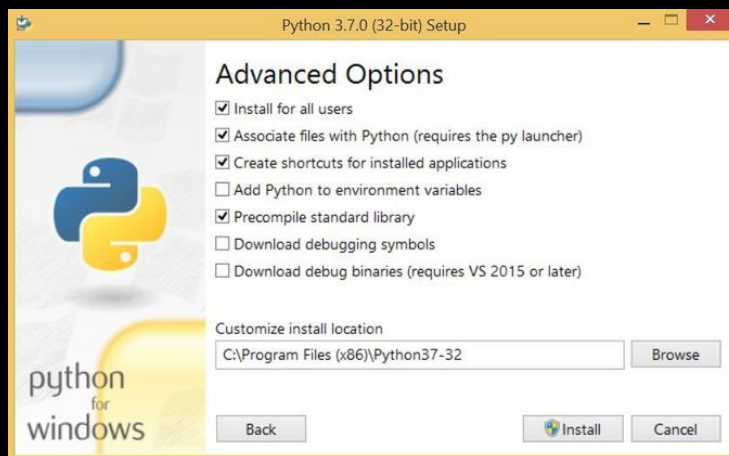
❖ **Technical strength of Python**

- *It's Object-Oriented*
- *It's Free*
- *It's Portable*
- *It's Powerful*
- *It's Mixable*
- *It's Easy to Use*
- *It's Easy to Learn*

Python Installation

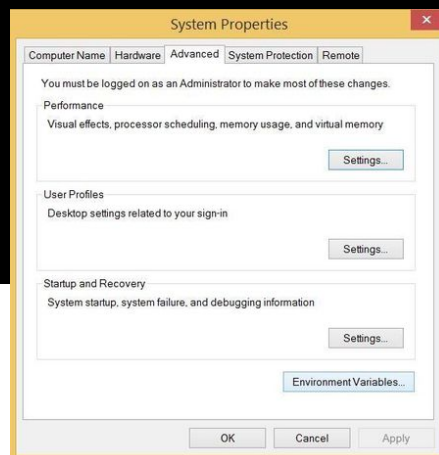
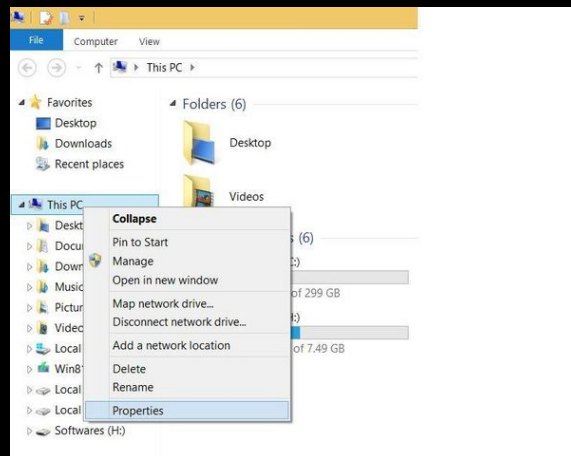


3. The following window shows all the optional features. All the features need to be installed and are checked by default; we need to click next to continue.

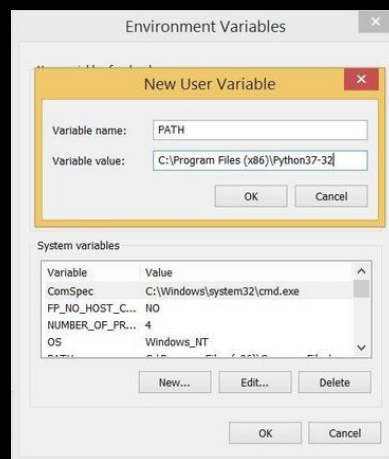
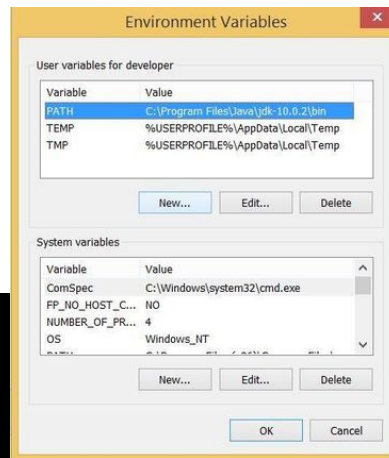


ons
check-

6. Now, try to run python on the command prompt. (press window button + R)
Type the command python in case of python2 or python3 in case of python3. It will show an error as given in the below image. It is because we haven't set the path.

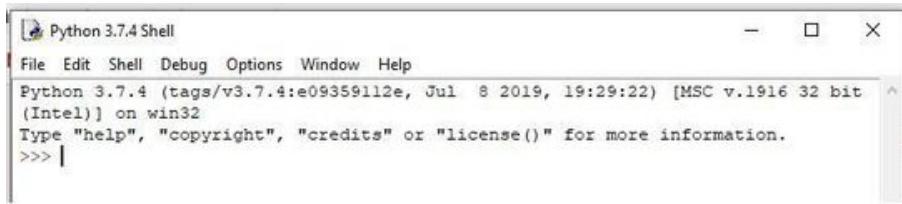


8. Add the new path variable in the user variable section.

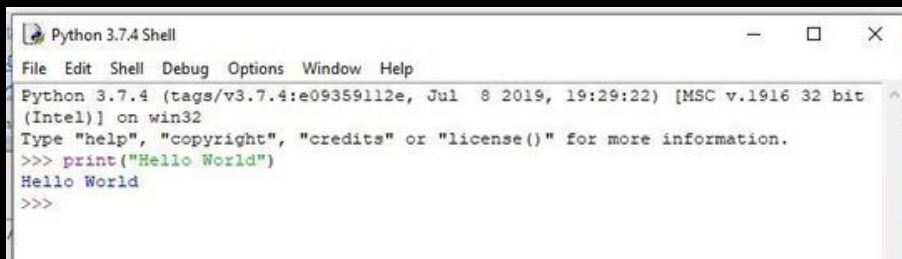


...one at the interactive prompt. It is preferable in the case where we are concerned about the output of each line of our Python program.

- To open the interactive mode, open the terminal (or command prompt) and type `python` (`python3` in case if you have Python2 and Python3 both installed on your system).
- It will open the following prompt where we can execute the Python statement and check their impact on the console.



```
Python 3.7.4 Shell
File Edit Shell Debug Options Window Help
Python 3.7.4 (tags/v3.7.4:e09359112e, Jul 8 2019, 19:29:22) [MSC v.1916 32 bit
(Intel)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> |
```



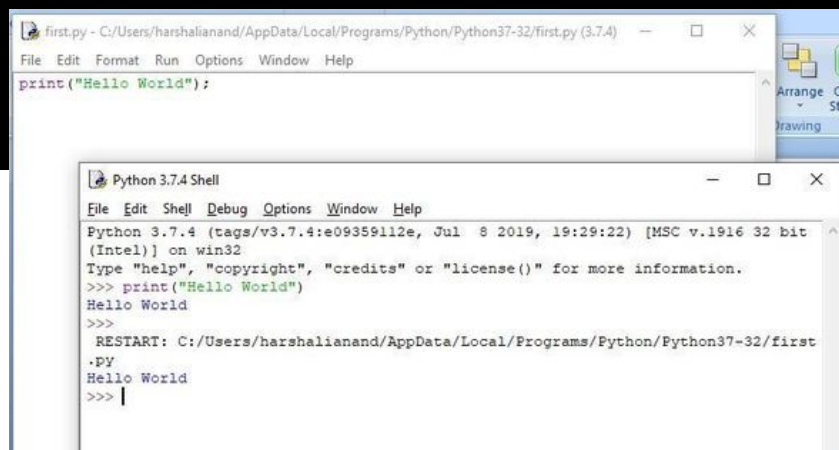
```
Python 3.7.4 Shell
File Edit Shell Debug Options Window Help
Python 3.7.4 (tags/v3.7.4:e09359112e, Jul 8 2019, 19:29:22) [MSC v.1916 32 bit
(Intel)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> print("Hello World")
Hello World
>>>
```

2

However,

For this
create a
de in it.

nd on the



```
first.py - C:/Users/harshalianand/AppData/Local/Programs/Python/Python37-32/first.py (3.7.4)
File Edit Format Run Options Window Help
print("Hello World");

Python 3.7.4 Shell
File Edit Shell Debug Options Window Help
Python 3.7.4 (tags/v3.7.4:e09359112e, Jul 8 2019, 19:29:22) [MSC v.1916 32 bit
(Intel)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> print("Hello World")
Hello World
>>>
RESTART: C:/Users/harshalianand/AppData/Local/Programs/Python/Python37-32/first
.py
Hello World
>>> |
```


❖ *Execute the following task using the basic knowledge of Python Language*

1. *Python Program to Print Hello MGM CET*
2. *Python Program to Check If number is Even or Odd*
3. *Python program to check whether the input character is an alphabet*
4. *Python Program to Check if a Number is Positive Negative or Zero*
5. *Python Program to Check Whether a String is Palindrome or Not*

Conclusion:

Result:

Industry:

-
-
-
-
-
-
-
-

Questions:

1. *How to print Hello MGM CET using Python?*

2. *Enlist the applications of Python.*

3. What is IDE?

4. State the shortcut to execute the Python code while using script file.

5. W

6. W

7. Explain the steps to create a Python project.

8. S

9. W

10. I

Experiment No. 2

Name of Student			
Roll No.			
DOP	DOS	Marks/Grade	Sign

Aim: Exploring basics of python like data types (strings, list, array, dictionaries, set, tuples) and control statements

Software Required: Python3.9 / Online Editor

Theo

(i) N

Outp

Outp

Outp

(ii) P

Outp

Outp

```
print("a[5:] = ", a[5:])
```

Output:

(iii) Python Tuple

```
t = (5,'program', 1+3j)
```

```
# t[1] = 'program'
```

```
print("t[1] = ", t[1])
```

Output:

```
# t[0:3] = (5, 'program', (1+3j))
```

```
print("t[0:3] = ", t[0:3])
```

Output:

(iv) List

Output:

Output:

Output:

Output:

```
# Generates error
```

```
# Strings are immutable in Python
```

```
s[5] ='d'
```

(v) Python Set

```
a = {5,2,3,1,4}
```

```
# printing set variable
```

```
print("a = ", a)
```

Output:

```
# data type of variable a
```

```
print(type(a))
```

Output:

(vi) List

Output:

Output:

Output:

Output:

(vii) Dictionary

Output:

Output:

```
>>> int(-10.6)
```

Output:

Conversion to and from string must contain compatible values.

```
>>> float('2.5')
```

Output:

```
>>> str(25)
```

Output:

```
>>> int('1p')
```

Output:

Output:

Output:

Output:

Output:

Output:

Conc

Synt

Consider the example below:

```
X = 10
```

```
Y = 12
```

```
if X < Y: print('X is less than Y') elif X > Y:
    print('X is greater than Y')
else:
    print('X and Y are equal')
```

Output:

While Loop

Here, the condition is checked first. If the condition is true, the statements are executed. If the condition is false, the loop is terminated. We are not sure how many times the loop will execute when we are using a while loop. Consider the following example:

Syntax:

```
while condition:
    code block to be executed
```

print

Output

For Loop

Like the while loop, the for loop is used to execute a block of code a number of times. The difference between the two is that in a for loop, the number of iterations depends on the condition. You will get a better idea about the difference between the two by looking at the syntax:

Syntax:

```
for variable in Sequence:
    statements
```

Notice here, we have specified the range, that means we know the number of times the code block will be executed.

Consider the example:

```
fruits = ['Banana', 'Apple', 'Grapes']
```

```
for index in range(len(fruits)):
    print (fruits[index])
```

Output:

Nested Loops

It follows that $\mathcal{C}_1$ is $\mathcal{C}_2$ -versal. Conversely,

Even

Cons

Output

Conc

Results

Experiment No. 3

Name of Student			
Roll No.			
DOP	DOS	Marks/Grade	Sign

Aim: Creating functions, classes and objects using python. Demonstrate exception handling and inheritance.

Software Required: Python3.9 / Online Editor

Theory

Python

Functions are a way of organizing code. They allow you to break down a large program into smaller, reusable pieces. The function is a block of organized, reusable code that is used in your main program by calling the function's name.

Python functions are blocks of organized, reusable code that is used in your main program by calling the function's name. The function is a block of organized, reusable code that is used in your main program by calling the function's name. The function is a block of organized, reusable code that is used in your main program by calling the function's name.

The function is a block of organized, reusable code that is used in your main program by calling the function's name. The function is a block of organized, reusable code that is used in your main program by calling the function's name. The function is a block of organized, reusable code that is used in your main program by calling the function's name.

Python functions are blocks of organized, reusable code that is used in your main program by calling the function's name. The function is a block of organized, reusable code that is used in your main program by calling the function's name. The function is a block of organized, reusable code that is used in your main program by calling the function's name.

There are

○ Functions are used to break down a large program into smaller, reusable pieces.

○ Functions are used to break down a large program into smaller, reusable pieces.

There are

○ Functions are used to break down a large program into smaller, reusable pieces.

○ Functions are used to break down a large program into smaller, reusable pieces.

○ We can track a large Python program easily when it is divided into multiple functions.

○ Reusability is the main achievement of Python functions.

○ However, Function calling is always overhead in a Python program.

Creating a Function

Python provides the **def** keyword to define the function. The syntax of the define function is given below.

Syntax:

```
def my_function(parameters):  
    function_block
```

Let's

o

o

o

o

o

the same

return

Func

In Py

defin

funct

must be

l the

Cons

ld".

Outp

Arguments in function

The arguments are types of information which can be passed into the function. The arguments are specified in the parentheses. We can pass any number of arguments, but they must be separate them with a comma.

Types of arguments

There may be several types of arguments which can be passed at the time of function call.

1. Required arguments
2. Keyword arguments
3. Default arguments
4. Variable length arguments

1. Required arguments

Till now we have seen that a function call must have the same number of arguments as the function definition. If the number of arguments is not the same, then an error will be raised. If the arguments are not the same, then an error will be raised. If the arguments are not the same, then an error will be raised.

2. Default arguments

Python allows us to provide default values for the arguments of the function. If the argument is not provided, then the default value will be used.

3. Variable length arguments

In Python, we can pass a variable number of arguments to a function. This is done by using an asterisk (*) before the argument name. This allows us to pass a variable number of arguments to a function.

How to pass a variable number of arguments to a function? We can use the asterisk (*) before the argument name. This is done by using an asterisk (*) before the argument name.

4. Keyword arguments

Python allows us to pass arguments to a function by using keywords. This is done by using an equals sign (=) before the argument name. This allows us to pass arguments to a function by using keywords.

The name of the arguments is treated as the keywords and matched in the function calling and definition. If the same match is found, the values of the arguments are copied in the function definition.

Questions:

1. What are the various Python language editor available for writing Python code?

2. Er

3. Er

4. Ca

5. W

6. Co

7. Write in short about Python language.

8. Enlist various real life applications of Python language

9. W

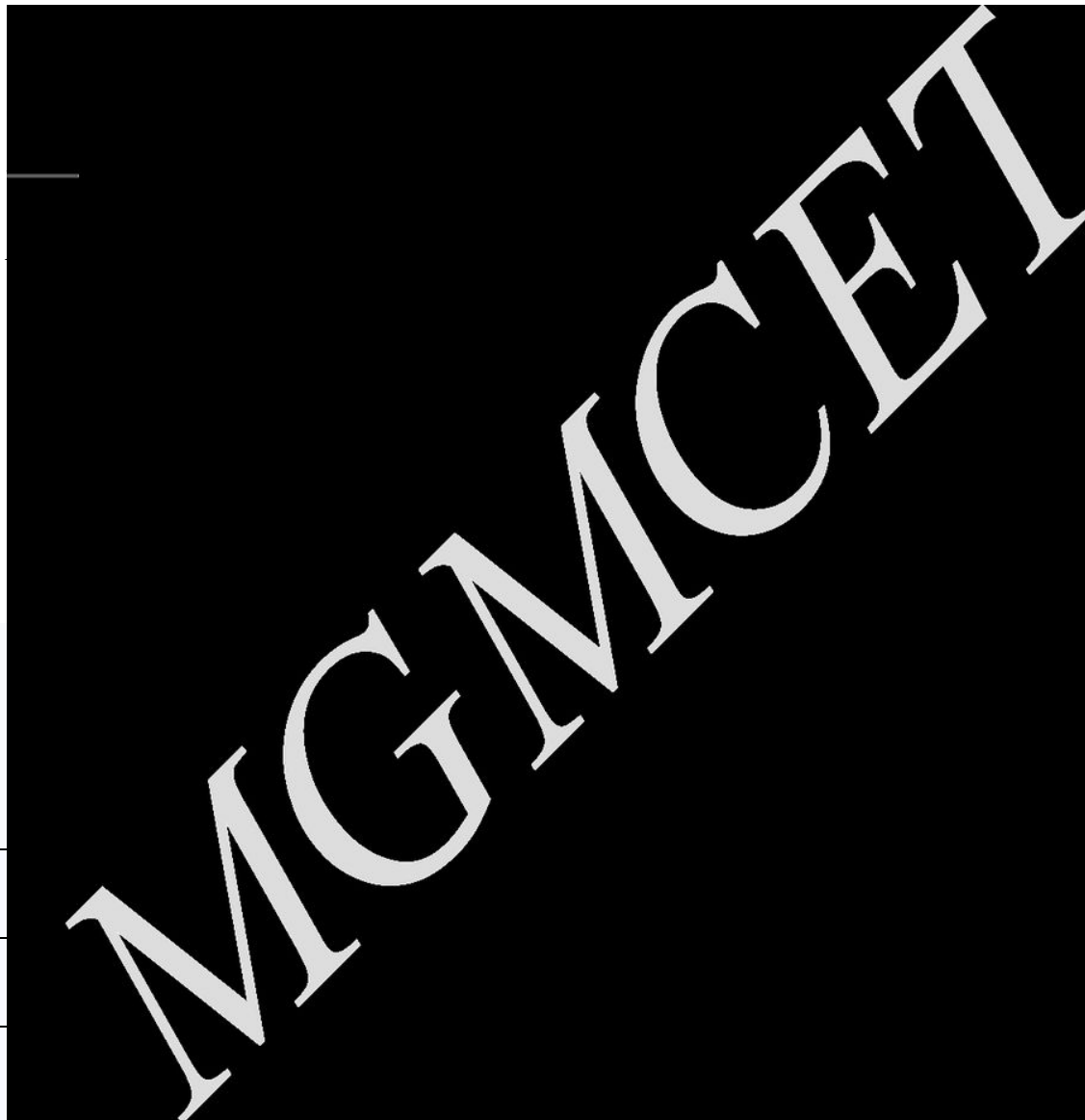
10. W

Experiment No. 4

Name of Student			
Roll No.			
DOP	DOS	Marks/Grade	Sign

Aim: Exploring Files and directories

- 4.a. Python program to append data to existing file and then display the entire file
- 4.b. Python program to count number of lines, words and characters in a file
- 4.c. Python program to display file available in current directory .



used to
Memory
for future

are done,

n the

nd

<code>fileno()</code>	Returns an integer number (file descriptor) of the file.
-----------------------	--

<code>flush()</code>	Flushes the write buffer of the file stream.
<code>isatty()</code>	Returns <code>True</code> if the file stream is interactive.
<code>read(n)</code>	Reads at most <code>n</code> characters from the file. Reads till end of file if it is negative or <code>None</code> .
<code>readable()</code>	Returns <code>True</code> if the file stream can be read from.
<code>readline(n=-1)</code>	Reads and returns one line from the file. Reads in at most <code>n</code> bytes if specified.
<code>readlines(n=-1)</code>	Reads and returns a list of lines from the file. Reads in at most <code>n</code> lines if specified.
<code>seek(offset, whence=0)</code>	Changes the file's current position (the reference point) to the position <code>offset</code> bytes away from the reference point <code>whence</code> . (start, 0) means from the beginning of the file (the default), (start, 1) means from the current position, and (start, 2) means from the end of the file.
<code>seek(offset, whence=0)</code>	Changes the file's current position (the reference point) to the position <code>offset</code> bytes away from the reference point <code>whence</code> . (start, 0) means from the beginning of the file (the default), (start, 1) means from the current position, and (start, 2) means from the end of the file.
<code>tell()</code>	Returns the current file position, as a byte offset from the beginning of the file.
<code>truncate(size=None)</code>	Truncates the file to the size <code>size</code> bytes. If <code>size</code> is not specified, the file is truncated to its current position.
<code>write(s)</code>	Writes the string <code>s</code> to the file.
<code>writeln(s)</code>	Writes the string <code>s</code> to the file, followed by a newline character.
<code>writelines(lines)</code>	Writes the list of strings <code>lines</code> to the file.

Python Directory:

If there are a large number of files to handle in our Python program, we can arrange our code within different directories to make things more manageable.

A directory or folder is a collection of files and subdirectories. Python has the os module that provides us with many useful methods to work with directories (and files as well).

All files are contained within various directories, and Python has no problem handling these too. The os module has several methods that help you create, remove, and change directories.

1. The mkdir() Method

You can use the mkdir() method of the os module to create directories in the current directory. You need to supply an argument to this method which contains the name of the directory to be created.

Syntax

os.mkdir()

Example

Follow

```
#!/usr/bin/python
```

```
import os
```

```
# Create a directory
```

```
os.mkdir('new_dir')
```

2. The chdir() Method

You can use the chdir() method of the os module to change the current directory. It takes an argument which contains the name of the directory to be changed to.

Syntax

os.chdir()

Example

Follow

```
#!/usr/bin/python
```

```
import os
```

```
# Change the current directory
```

```
os.chdir('new_dir')
```

3. The getcwd() Method

The getcwd() method of the os module returns the current working directory as a string.

Syntax

`os.getcwd()`

Example

Following is the example to give current directory –

```
#!/usr/bin/python
```

```
import os
```

```
# This would give location of the current directory
```

```
os.getcwd()
```

4. The rmdir() Method

The rmdir() method deletes the directory, which is passed as an argument in the method.

Before removing a directory, all the contents in it should be removed.

Syntax

```
os.rmdir('dirname')
```

Example

Following is the example to remove a directory. The directory should be a qualified name.

```
#!/usr/bin/python
```

```
import os
```

```
# This would give location of the current directory
```

```
os.rmdir('dirname')
```

Conclusion

Result

Experiment No. 5

Name of Student			
Roll No.			
DOP	DOS	Marks/Grade	Sign

Aim: Creating GUI with python containing widgets such as labels, textbox, radio, checkboxes and custom dialog boxes

Software Required: Python3.9 / Online Editor

Theory

Python is a
important

1. Tkinter

2. wxPython

3. JPython
Java

Tkinter

fast

inter

Cre

follo

➤

➤

➤

➤

Exam

#!/us

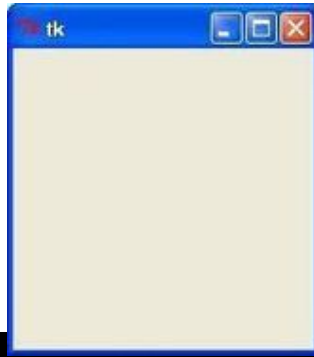
imp

top = Tkinter.Tk()

Code to add widgets will go here...

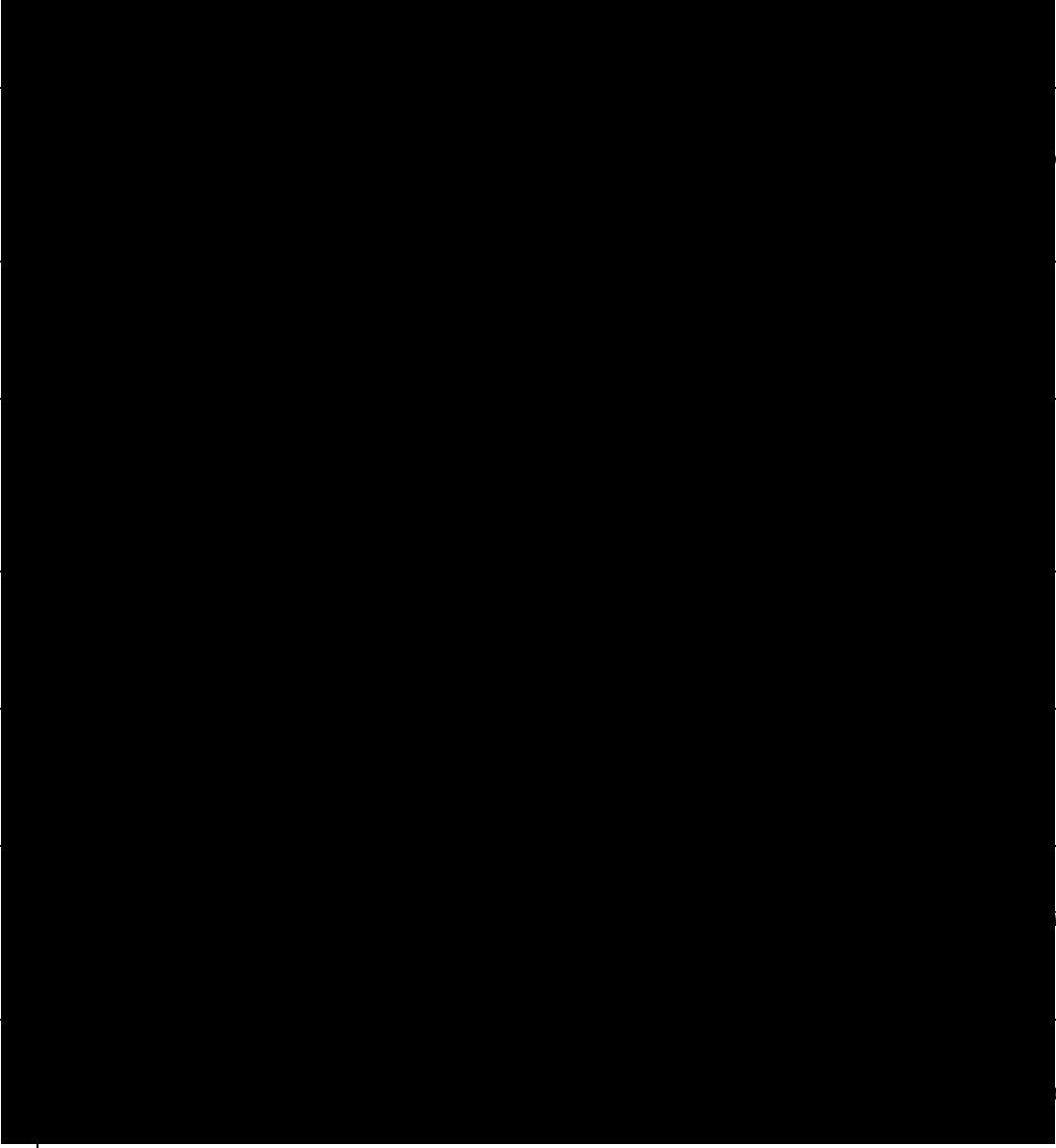
top.mainloop()

This would create a following window –



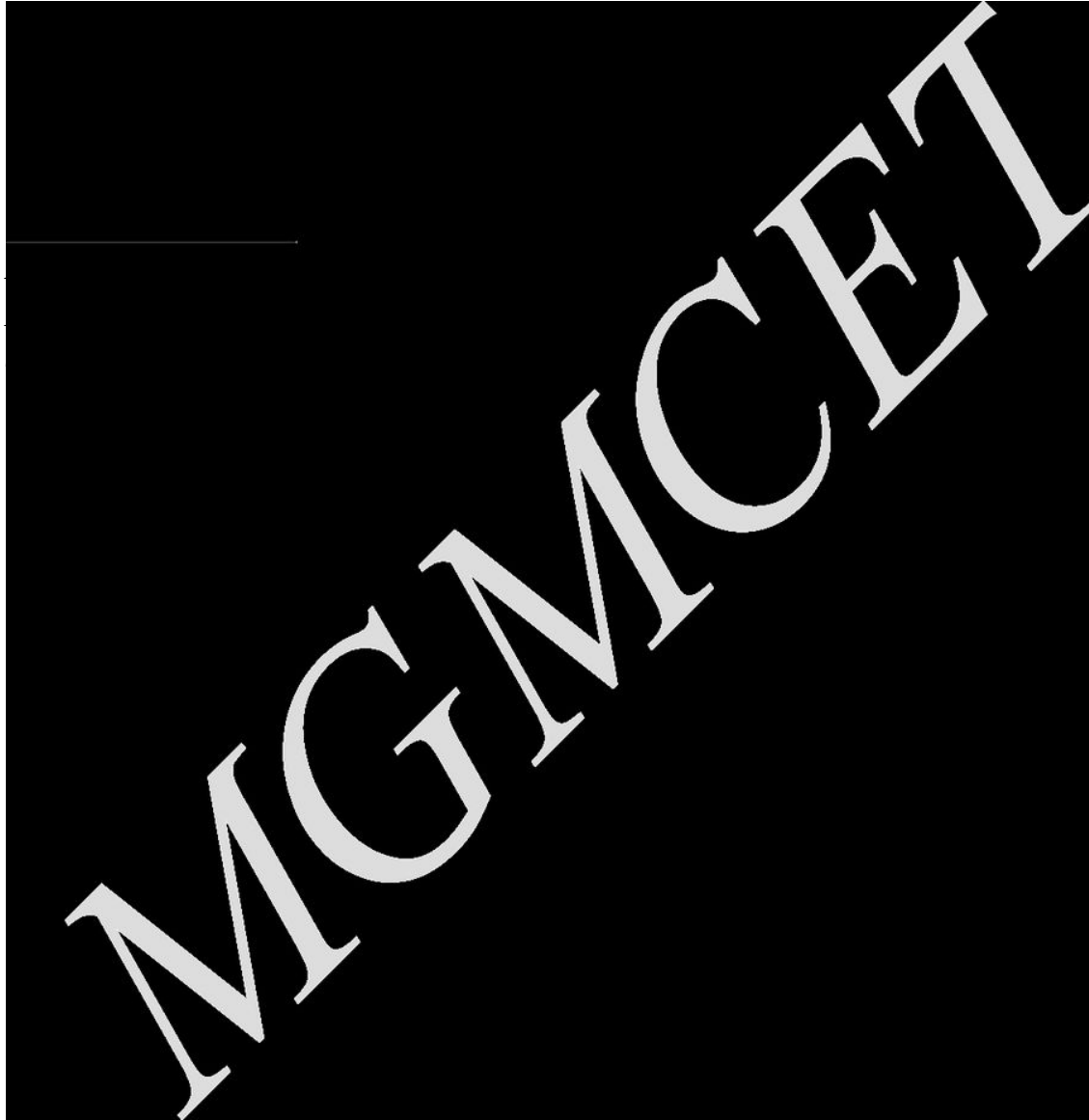
Tkin
Tkin
appl
Ther

Sr.No	
1	
2	rectangles, in
3	The user can
4	from a user.
5	
6	It can also
	contain images.
7	<i>Listbox</i> The Listbox widget is used to provide a list of options to a user.
8	<i>Menubutton</i>

	<i>The Menubutton widget is used to display menus in your application.</i>
9	<i>Menu</i> <i>The Menu widget is used to provide various commands to a user. These commands are contained inside Menubutton.</i>
10	<i>Message</i>  from a user.
11	ns. The user
12	
13	such as list
14	
15	
16	in be used to
17	es, arranged
18	<i>LabelFrame</i> <i>A labelframe is a simple container widget. Its primary purpose is to act as a spacer or container for complex window layouts.</i>
19	<i>tkMessageBox - This module is used to display message boxes in your applications.</i>

Standard attributes

- *Dimensions*
- *Colors*
- *Fonts*
- *Anchors*



have the
following

e placing

ucture in

hem in a

Experiment No. 6

Name of Student			
Roll No.			
DOP	DOS	Marks/Grade	Sign

Aim: Menu driven program for data structure using built in function for link list, stack and queue

Software Required: Python3.9 / Online Editor

Theory

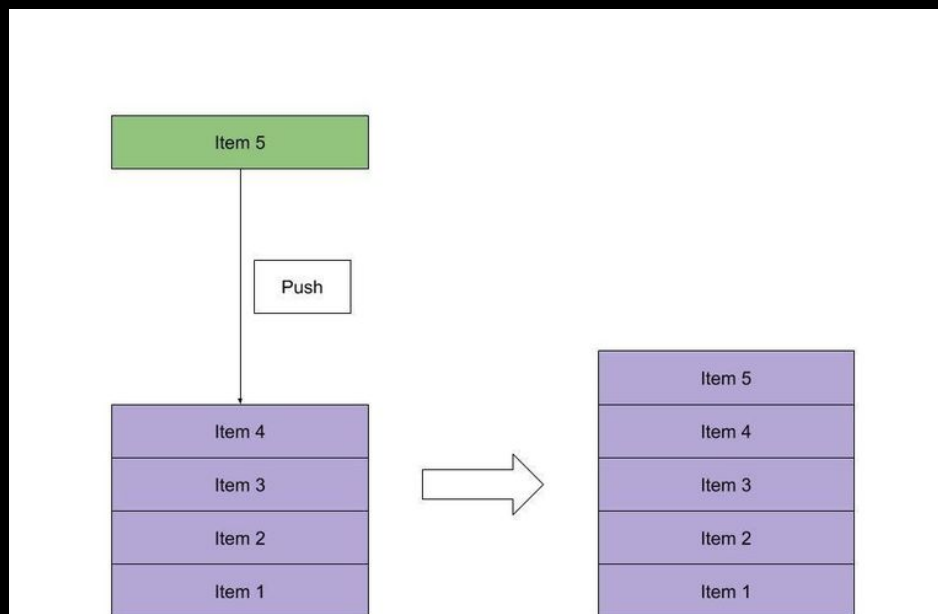
How

Stack

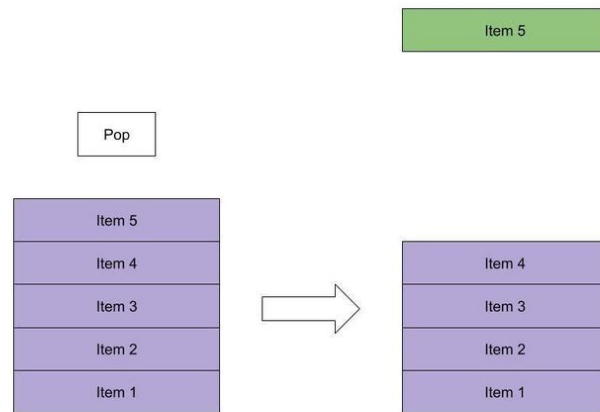
Stack is a linear data structure that follows the Last In First Out (LIFO) principle. It is like a stack of coins where the last coin added is the first to be removed.

To insert

•



- *pop* - removes the element at the top of the stack:



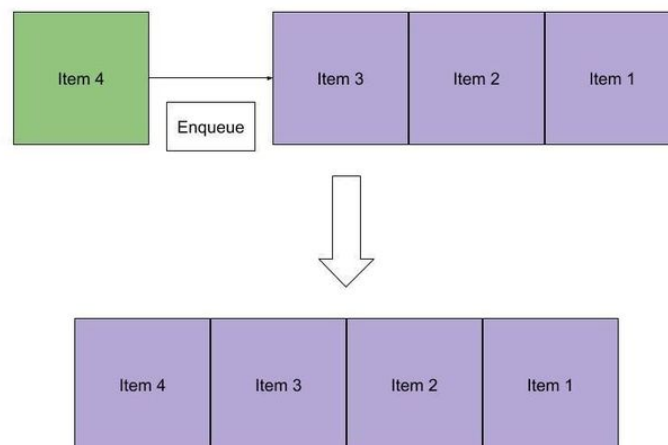
Queue

Queue is a data structure that follows the First In First Out (FIFO) principle. It is used to store elements in a queue and retrieve them in the same order.

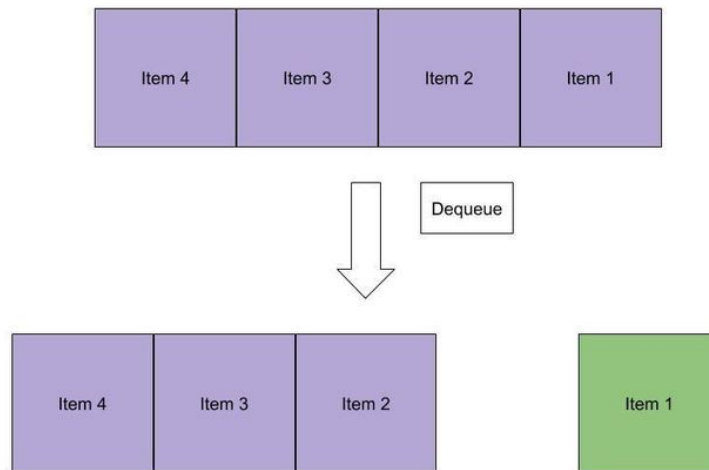
waiting in a queue and enjoy

To insert an element into a queue, we use the enqueue operation.

-



- *dequeue* - removes the element at the beginning of the queue:



Conc

Resu

Experiment No. 7

Name of Student			
Roll No.			
DOP	DOS	Marks/Grade	Sign

Aim: Program to demonstrate CRUD (create, read, update and delete) operations on database (SQLite/ MySQL) using python.

Software Required: Python3.9 / Online Editor

Theory

SQLite is a self-contained, serverless, zero-configuration database engine. It is a lightweight database engine that is easy to use and requires no special hardware or software. It is a good choice for applications that require a small, fast, and reliable database. SQLite is a good choice for applications that require a small, fast, and reliable database. It is a good choice for applications that require a small, fast, and reliable database.

SQLite is a self-contained, serverless, zero-configuration database engine. It is a lightweight database engine that is easy to use and requires no special hardware or software. It is a good choice for applications that require a small, fast, and reliable database. SQLite is a good choice for applications that require a small, fast, and reliable database. It is a good choice for applications that require a small, fast, and reliable database.

SQLite is a self-contained, serverless, zero-configuration database engine. It is a lightweight database engine that is easy to use and requires no special hardware or software. It is a good choice for applications that require a small, fast, and reliable database. SQLite is a good choice for applications that require a small, fast, and reliable database. It is a good choice for applications that require a small, fast, and reliable database.

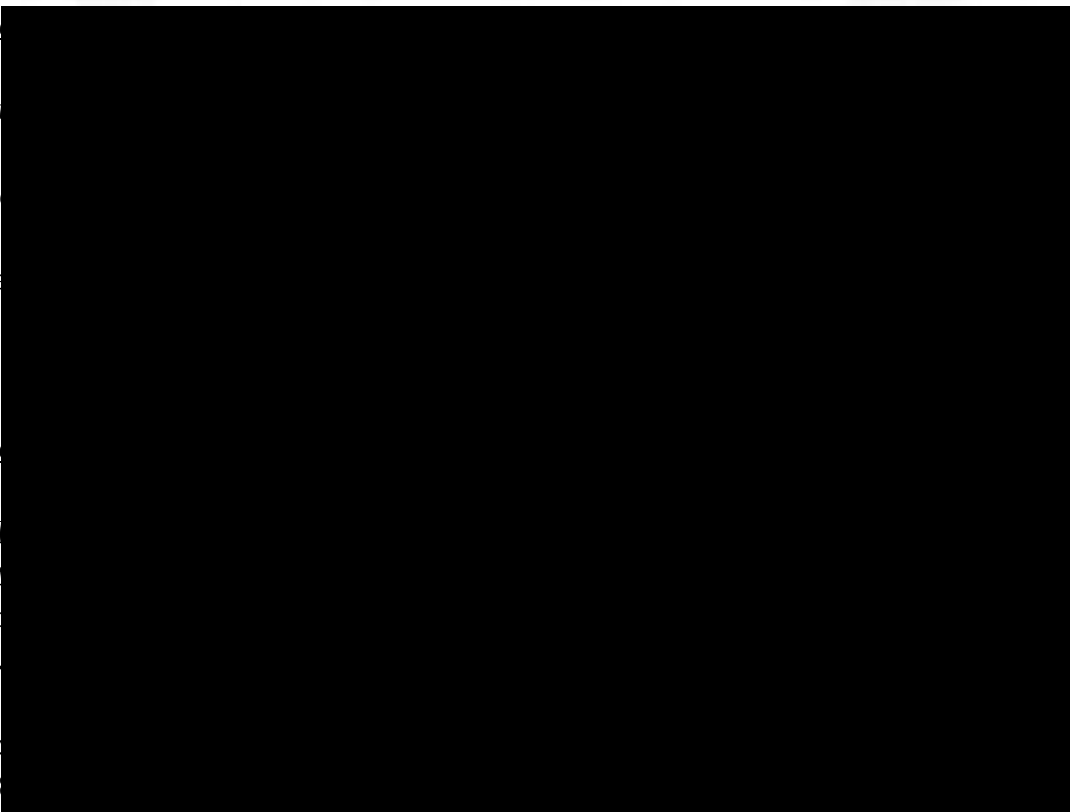
SQLite is a self-contained, serverless, zero-configuration database engine. It is a lightweight database engine that is easy to use and requires no special hardware or software. It is a good choice for applications that require a small, fast, and reliable database. SQLite is a good choice for applications that require a small, fast, and reliable database. It is a good choice for applications that require a small, fast, and reliable database.

SQLite is a self-contained, serverless, zero-configuration database engine. It is a lightweight database engine that is easy to use and requires no special hardware or software. It is a good choice for applications that require a small, fast, and reliable database. SQLite is a good choice for applications that require a small, fast, and reliable database. It is a good choice for applications that require a small, fast, and reliable database.

SQLite is a self-contained, serverless, zero-configuration database engine. It is a lightweight database engine that is easy to use and requires no special hardware or software. It is a good choice for applications that require a small, fast, and reliable database. SQLite is a good choice for applications that require a small, fast, and reliable database. It is a good choice for applications that require a small, fast, and reliable database.

```
C:\Windows\system32\cmd.exe
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Users\dell>D:
D:\>cd sqlite
D:\SQLite>
```

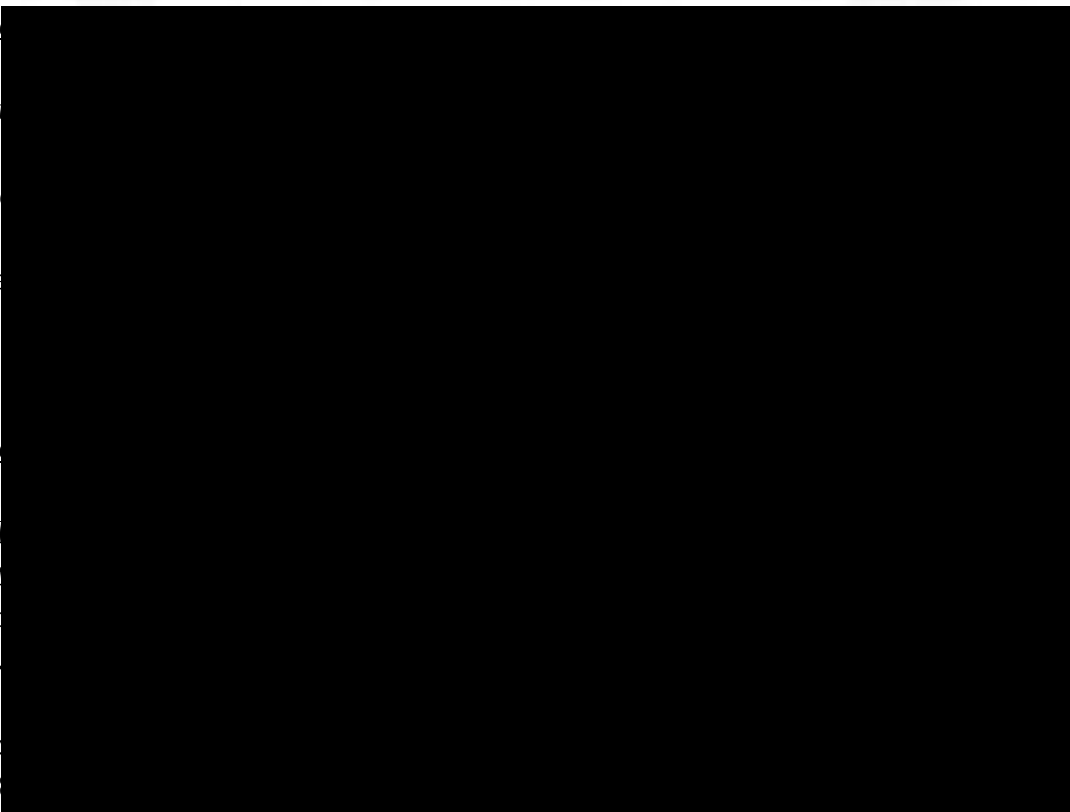
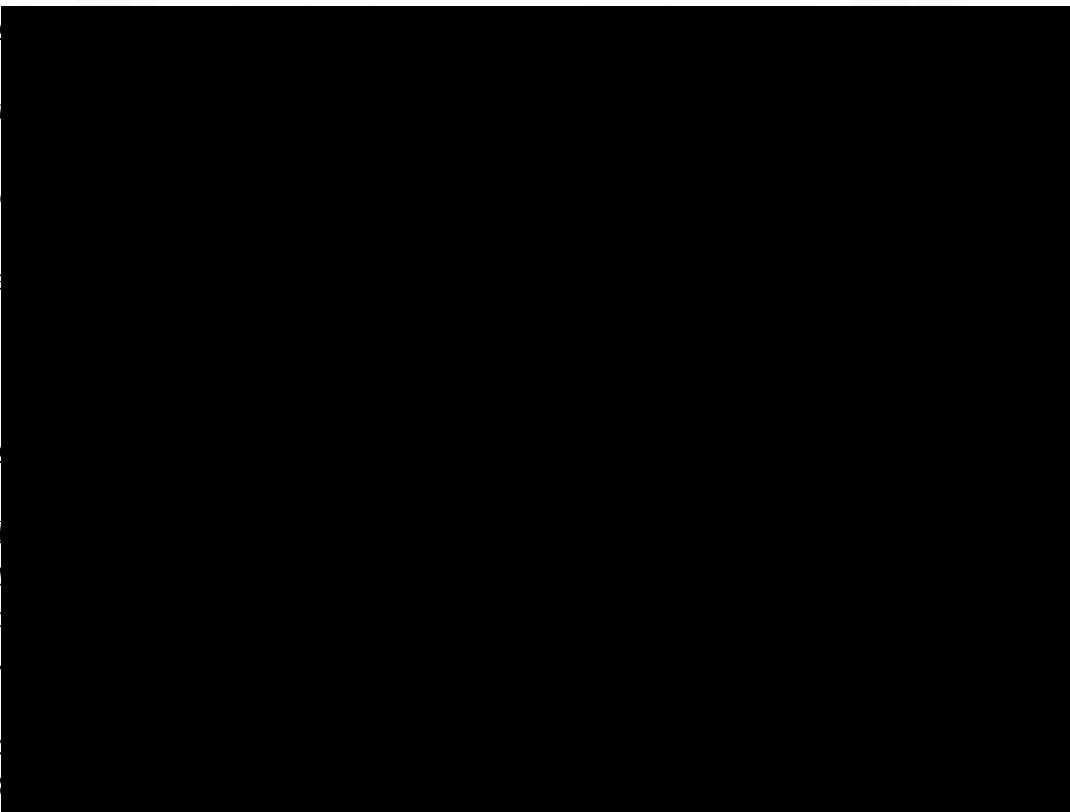
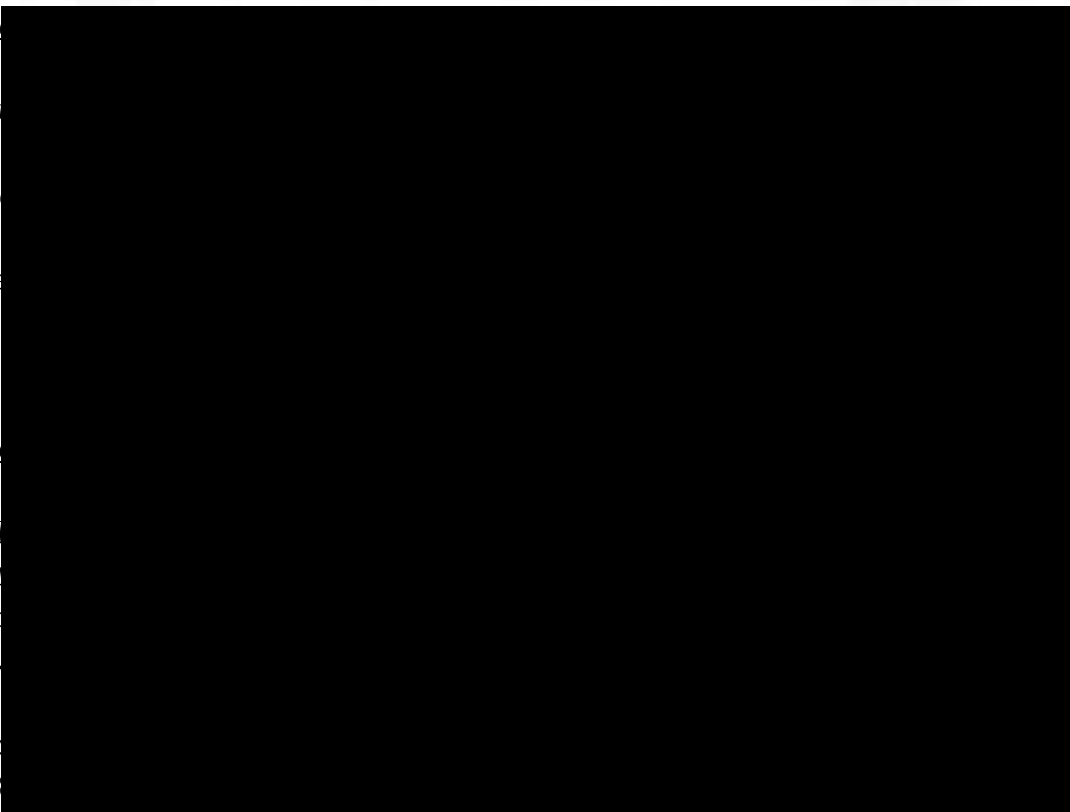
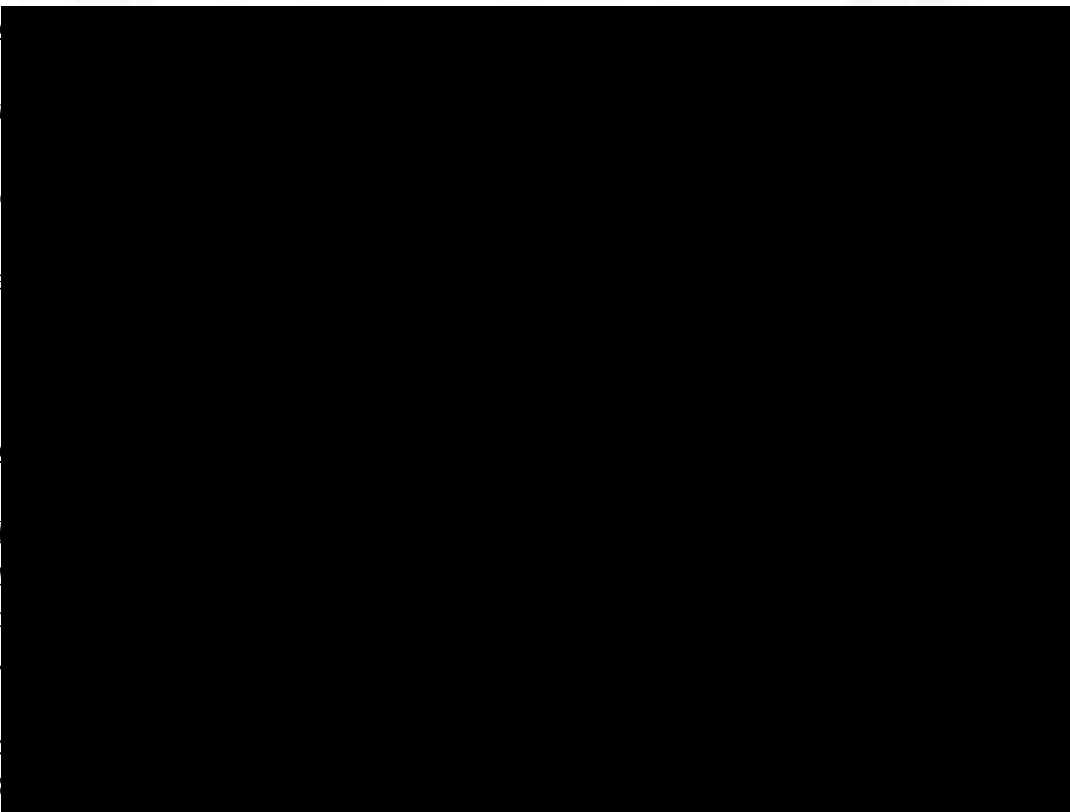
Sqlite  new table.

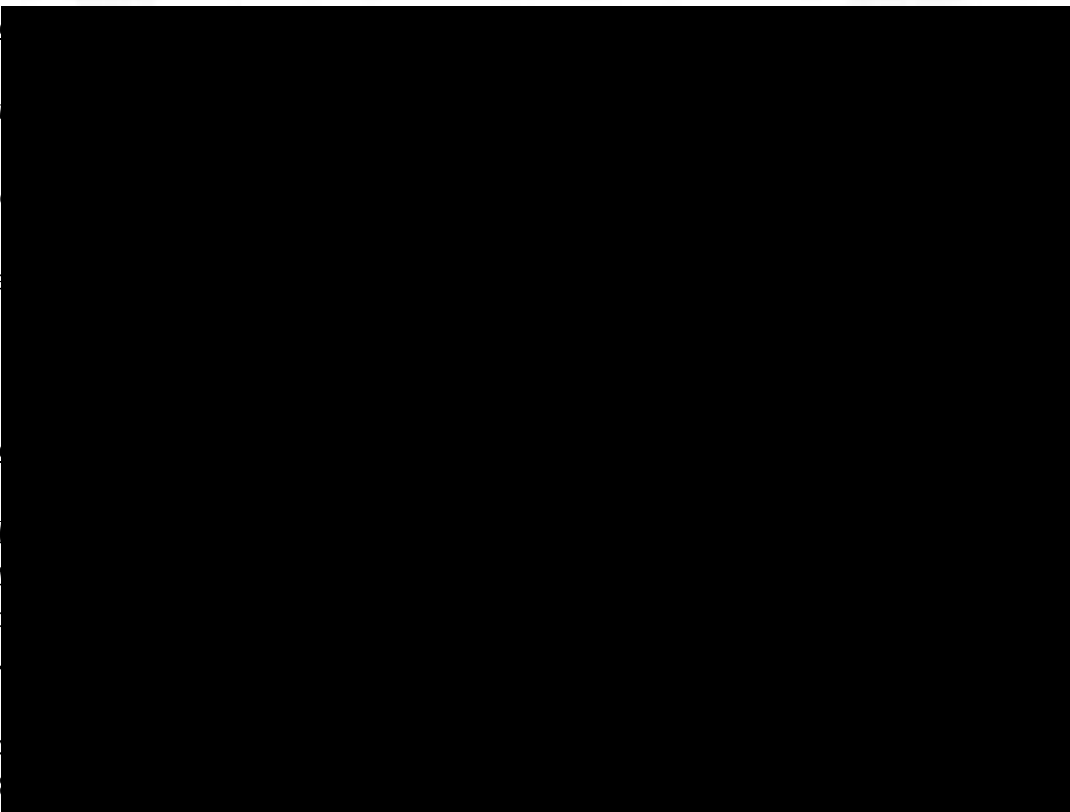
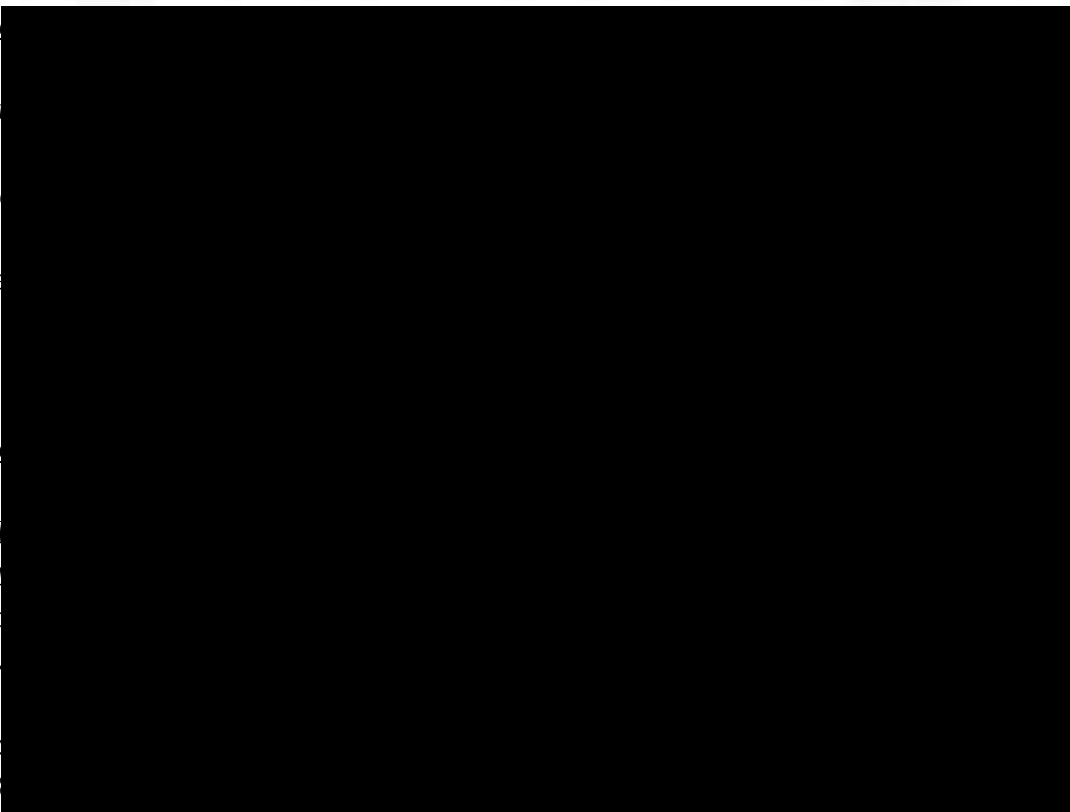
(Ens

Add

To re

Pyth

data  Group for
func  interfacing
stand  DB-API
249)  sal (PEP-

cont  activity. It
RDB  ng. Other

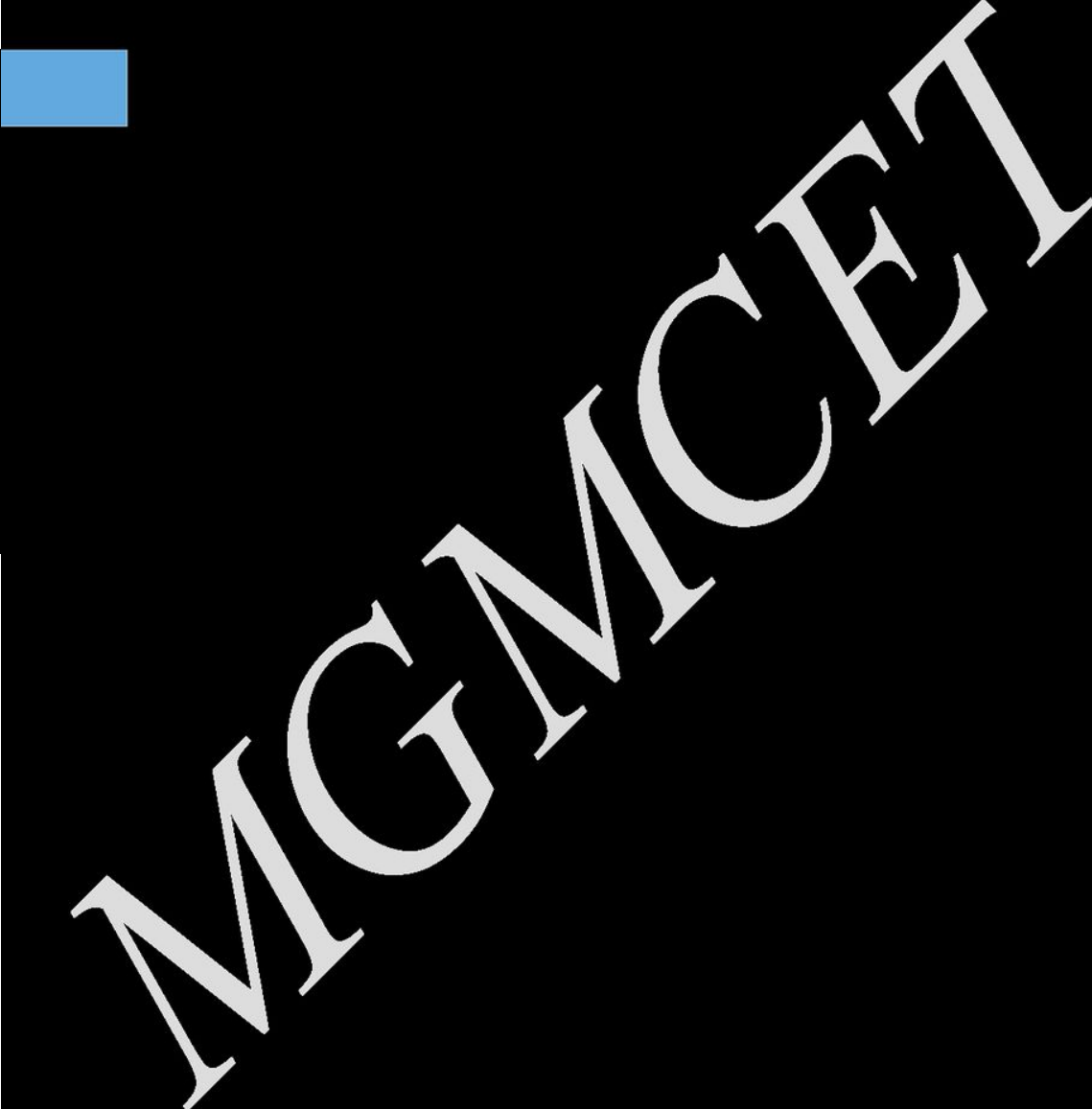
- MySQL: PyMySQL module
- Oracle: Cx-Oracle module
- SQL Server: PyMsSql module
- PostGreSQL: psycopg2 module
- ODBC: pyodbc module

As per the prescribed standards, the first step in the process is to obtain the connection to the object representing the database. In order to establish a connection with a SQLite database, sqlite3 module needs to be imported and the connect() function needs to be executed.

```
>>> import sqlite3
```

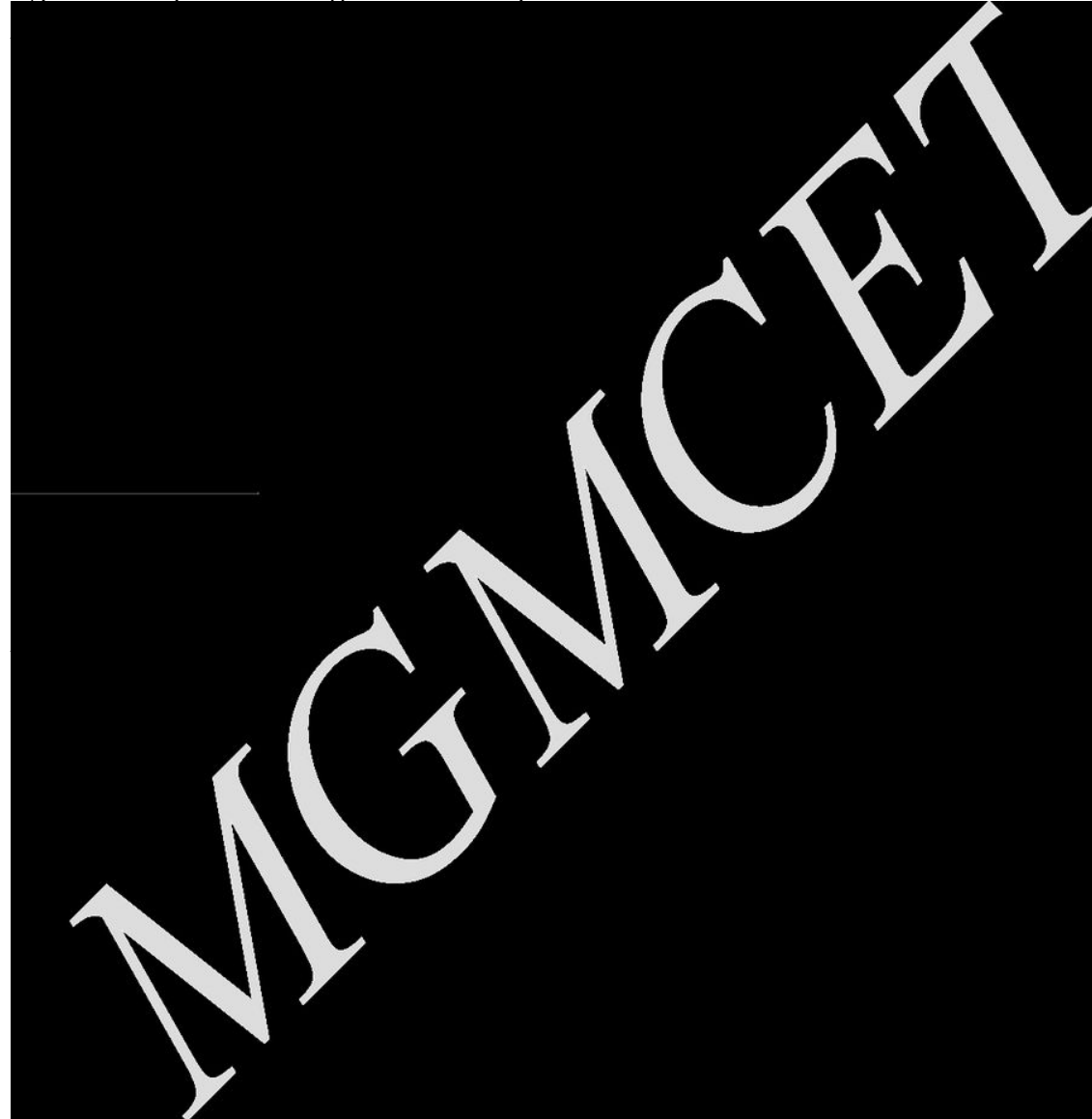
```
>>> db=sqlite3.connect('test.db')
```

The connect() function returns a connection object referring to the existing database or a new database if it doesn't exist.

		
		d be a no-
		int. It may
		ction after
		acts as a
		Hence, a
		following
fetchone()	Fetches the next row from the query result set.	
fetchall()	Fetches all remaining rows from the query result set.	
callproc()	Calls a stored procedure.	
close()	Closes the cursor object.	

The `commit()` and `rollback()` methods of the connection class ensure transaction control. The `execute()` method of the cursor receives a string containing the SQL query. A string having an incorrect SQL query raises an exception, which should be properly handled. That's why the `execute()` method is placed within the try block and the effect of the SQL query is persistently saved using the `commit()` method. If however, the SQL query fails, the resulting exception is processed by the except block and the pending transaction is undone using the `rollback()` method.

Typical use of the `execute()` method is as follows:



`db.rollback()`
`db.close()`

This can be verified using the `.tables` command in `sqlite` shell.

`E:\SQLite>sqlite3 test.db`

`SQLite version 3.25.1 2018-09-18 20:20:44`

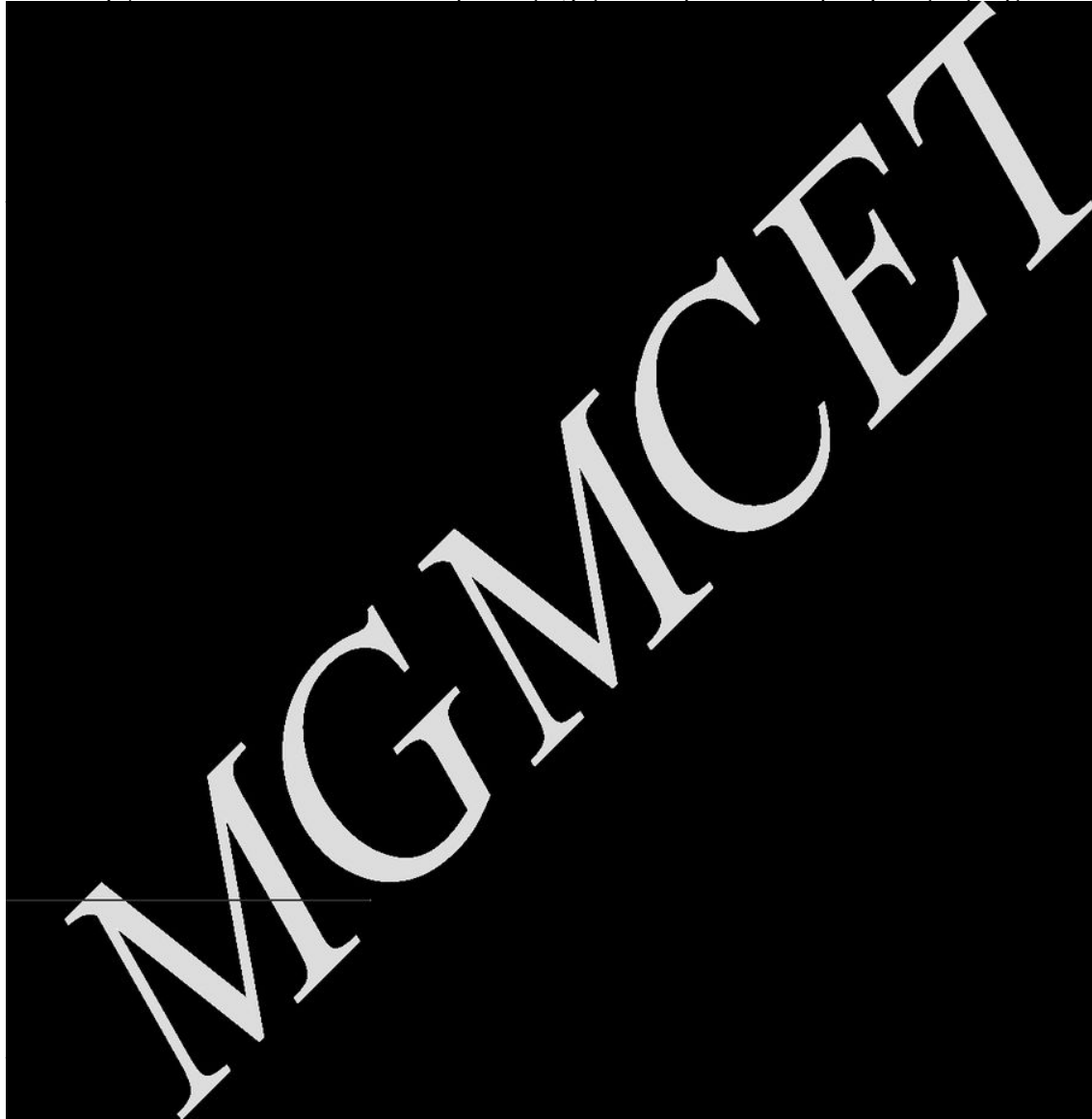
`Enter ".help" for usage hints.`

```
sqlite> .tables  
student
```

Insert a Record

Once again, the `execute()` method of the cursor object should be called with a string argument representing the `INSERT` query syntax. We have created a student table having three fields: name, age and marks. The string holding the `INSERT` query is defined as:

```
qry="INSERT INTO student (name, age, marks) VALUES ('Rajeev',20,50);"
```



possible
complete

way is to
y lead to
ended in
vides the
a record

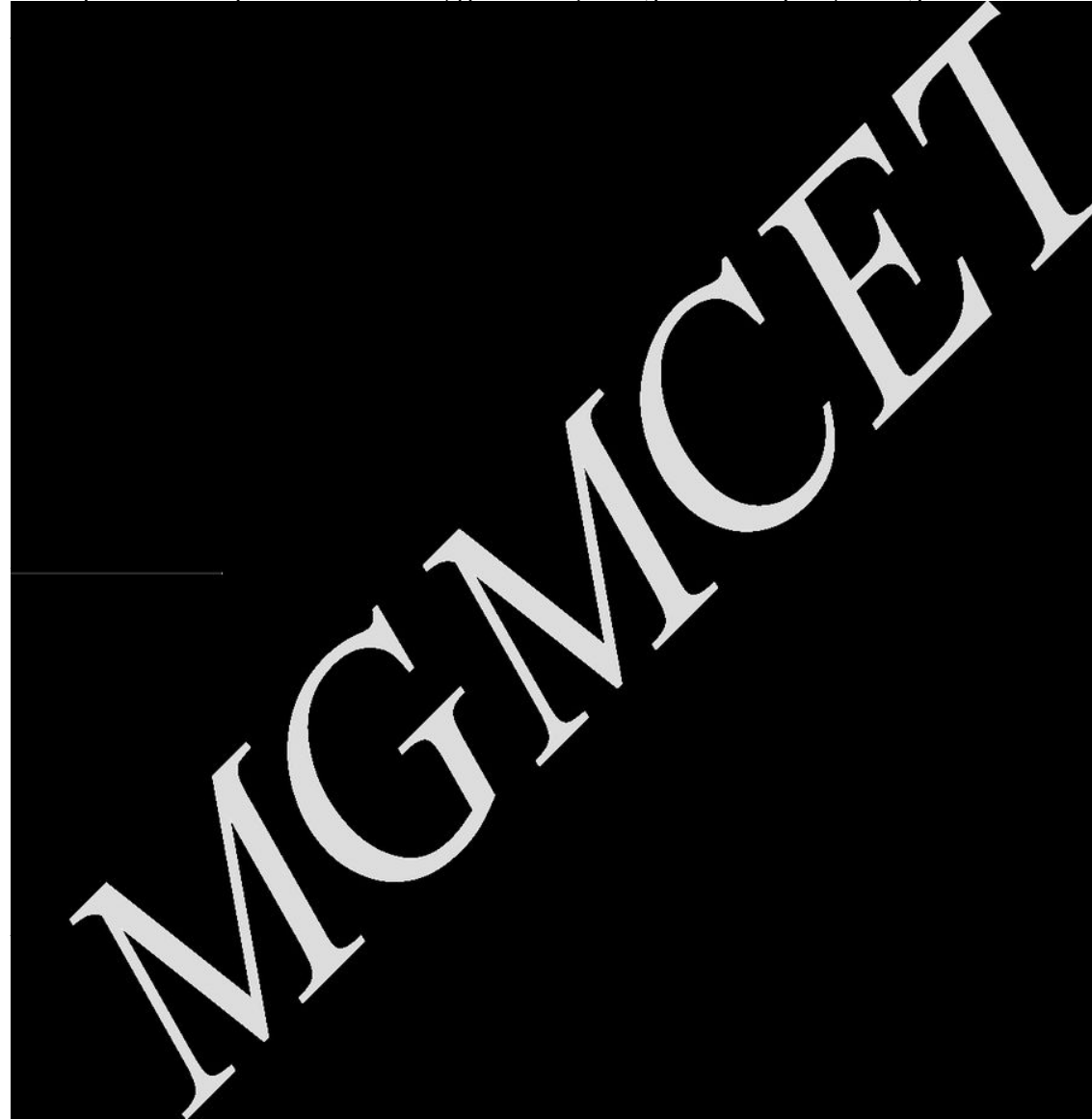
using the parameter substitution method:

Example:

```
import sqlite3  
db=sqlite3.connect('test.db')  
qry="insert into student (name, age, marks) values(?,?,?);"try:  
    cur=db.cursor()  
    cur.execute(qry, ('Vijaya', 16,75))  
    db.commit()
```

```
print ("one record added successfully")except:  
print("error in operation")  
db.rollback()  
db.close()
```

The `executemany()` method is used to add multiple records at once. Data to be added should be given in a list of tuples, with each tuple containing one record. The list object (containing tuples) is the parameter of the `executemany()` method, along with the query string.



```
cur.execute(sql)while True:  
    record=cur.fetchone()  
    if record==None:  
        break  
    print (record)  
db.close()
```

When executed, the following output is displayed in the Python shell:

Result:

(1, 'Rajeev', 20, 50.0)

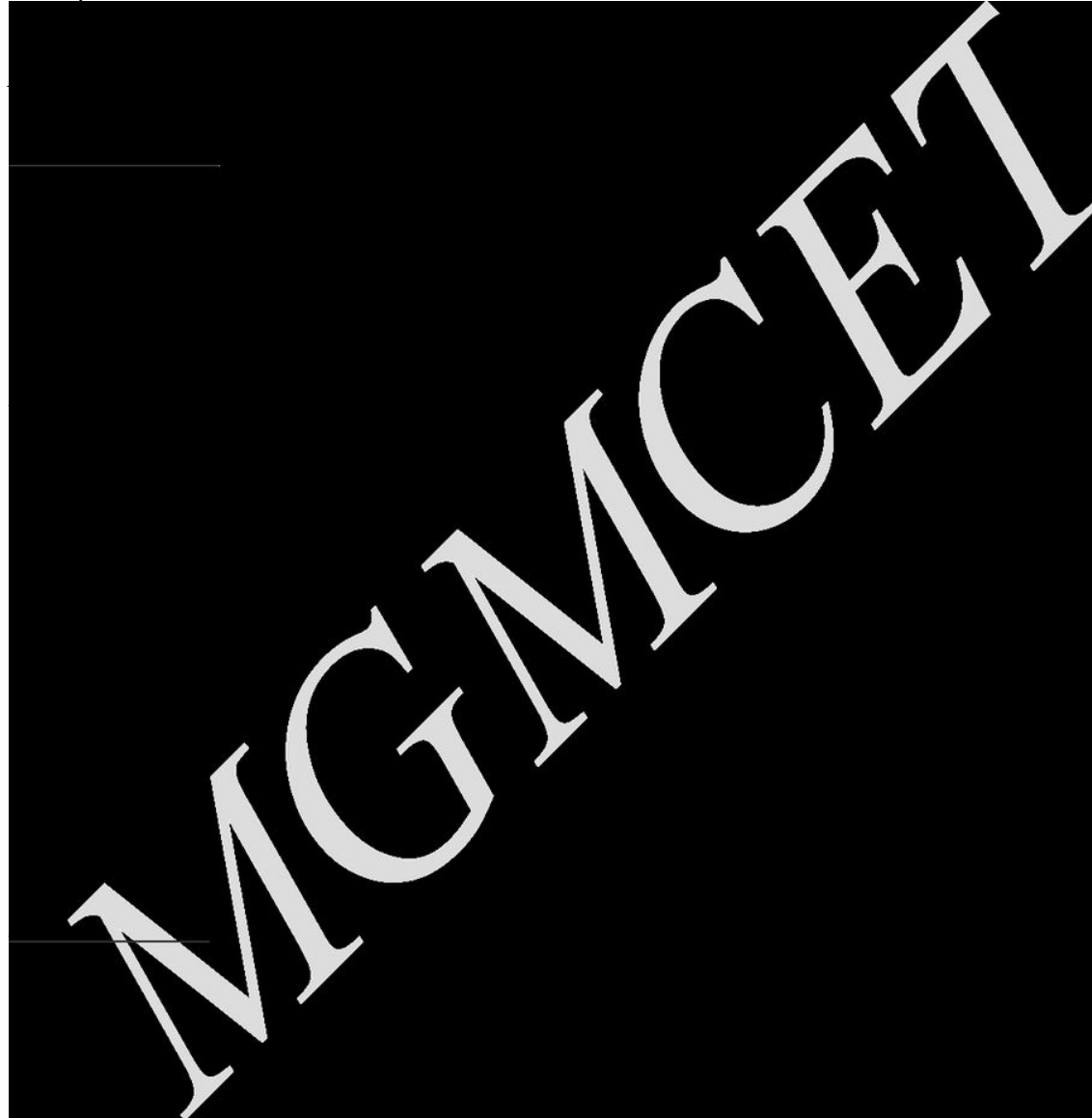
(2, 'Vijaya', 16, 75.0)

(3, 'Amar', 18, 70.0)

(4, 'Deepak', 25, 87.0)

The fetchall() method returns a list of tuples, each being one record.

Example:



yntax. To

low code

Example: Delete Record

```
import sqlite3
```

```
db=sqlite3.connect('test.db')
```

```
qry="DELETE from student where name=?;"try:
```

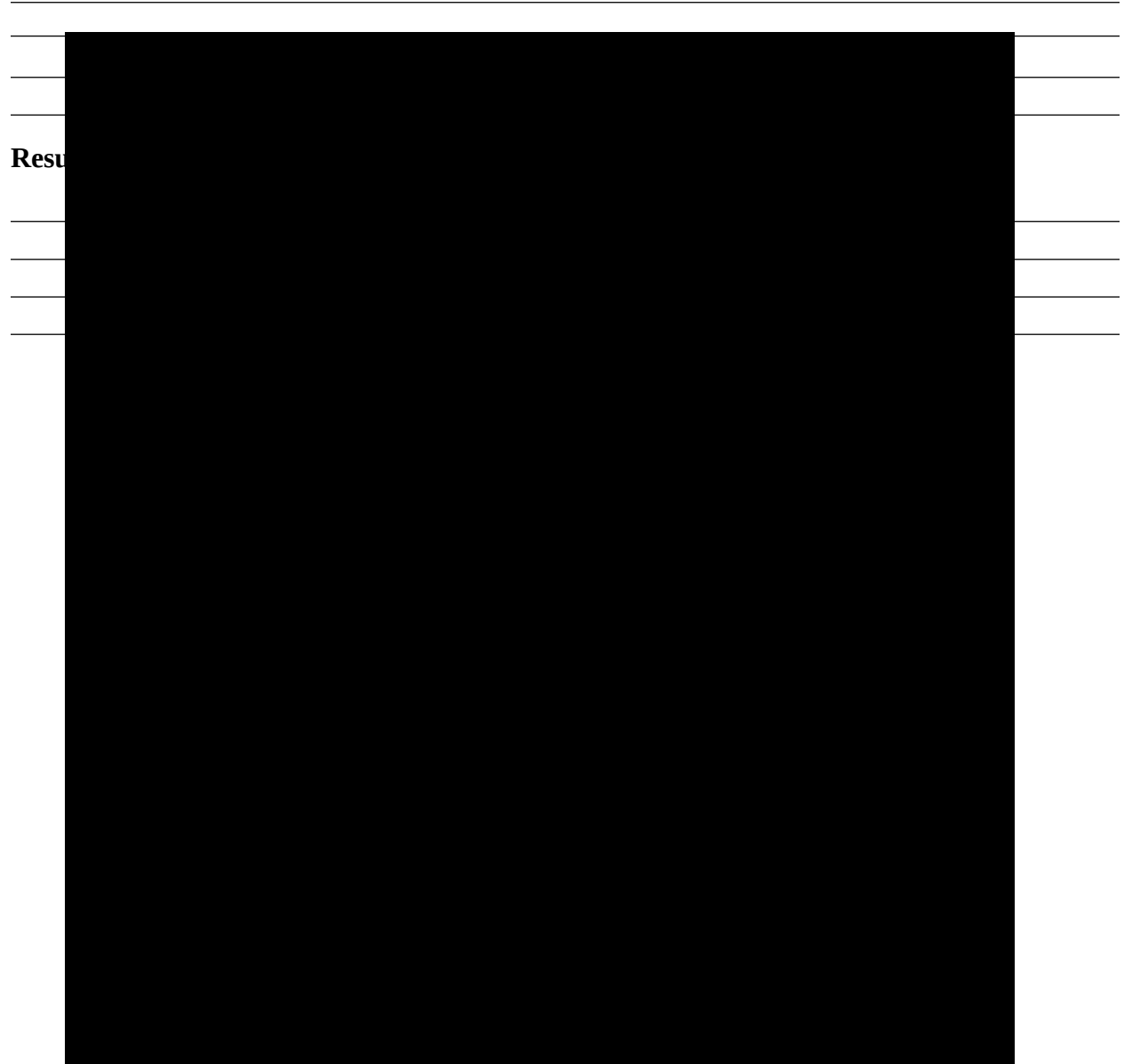
```
    cur=db.cursor()
```

```
    cur.execute(qry, ('Bill',))
```

```
    db.commit()
```

```
print("record deleted successfully")except:  
    print("error in operation")  
    db.rollback()  
db.close()
```

Conclusion:



Experiment No. 8

Name of Student			
Roll No.			
DOP	DOS	Marks/Grade	Sign

Aim: Creation of simple socket for basic information exchange between server and client.

Software Required: Python3.9 / Online Editor

Theory

To use

1. Socket

2. Socket

3. Socket

or process

resource

client

between

have

server

server

and to

The



will also terminate when client program terminates, this is optional and we can keep server program running indefinitely or terminate with some specific command in client request.

Python Socket Server

We will save python socket server program as socket_server.py. To use python socket connection, we need to import socket module. Then, sequentially we need to perform some task to establish connection between server and client.

We can obtain host address by using socket.gethostname() function. It is recommended to use port address above 1024 because port number lesser than 1024 are reserved for standard internet protocol.

See the below python socket server example code, the comments will help you to understand the code.

```
import socket
```

```
def server_program():
```

```
    # get local machine name
    host = socket.gethostname()
    port = 5000

    # create socket
    # TCP socket
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    # Bind the socket to the address and port
    s.bind((host, port))

    # Listen for incoming connections
    s.listen(1)

    while True:
        # Establish connection with client
        conn, addr = s.accept()

        # Receive the data from the client
        data = conn.recv(1024)

        # Decode the data
        data = data.decode('utf-8')

        # Print the data
        print(data)

        # Send the response back to the client
        response = 'Hello ' + data
        conn.send(response.encode('utf-8'))

        # Close the connection
        conn.close()

    if __name__ == '__main__':
        server_program()
```

request. If you want server to not quit when client connection is closed, just remove the if condition and break statement. Python while loop is used to run the server program indefinitely and keep waiting for client request.

Python Socket Client

We will save python socket client program as `socket_client.py`. This program is similar to the server program, except binding.

The main difference between server and client program is, in server program, it needs to bind host address and port address together.

See the below python socket client example code, the comment will help you to understand the code.

`import`

`def c`

`ho`

`po`

`cli`

`cli`

`me`

`wh`

`cli`

`if__`

`cli`

After `python3.6 socket_client.py` program.

last, `python3.6 socket_server.py` program. At

it worked on my test run of socket server and client example programs.

`pankaj$ python3.6 socket_server.py`

Connection from: ('127.0.0.1', 57822)

from connected user: Hi -> Hello

from connected user: How are you? -> Good

from connected user: Awesome! -> Ok then, bye!

`pankaj$`

pankaj\$ python3.6 socket_client.py -> Hi

Received from server: Hello -> How are you?

Received from server: Good -> Awesome!

Received from server: Ok then, bye! -> Byepankaj\$

Notice that socket server is running on port 5000 but client also requires a socket port to connect to the server. This port is assigned randomly by client connect call. In this case, it's 57822.

So, that's all for Python socket programming, python socket server and socket client example prog

Conc

Resu

Experiment No. 9

Name of Student			
Roll No.			
DOP	DOS	Marks/Grade	Sign

Aim: Creating web application using Django web framework to demonstrate functionality of user login and registration (also validating user detail using regular expression).

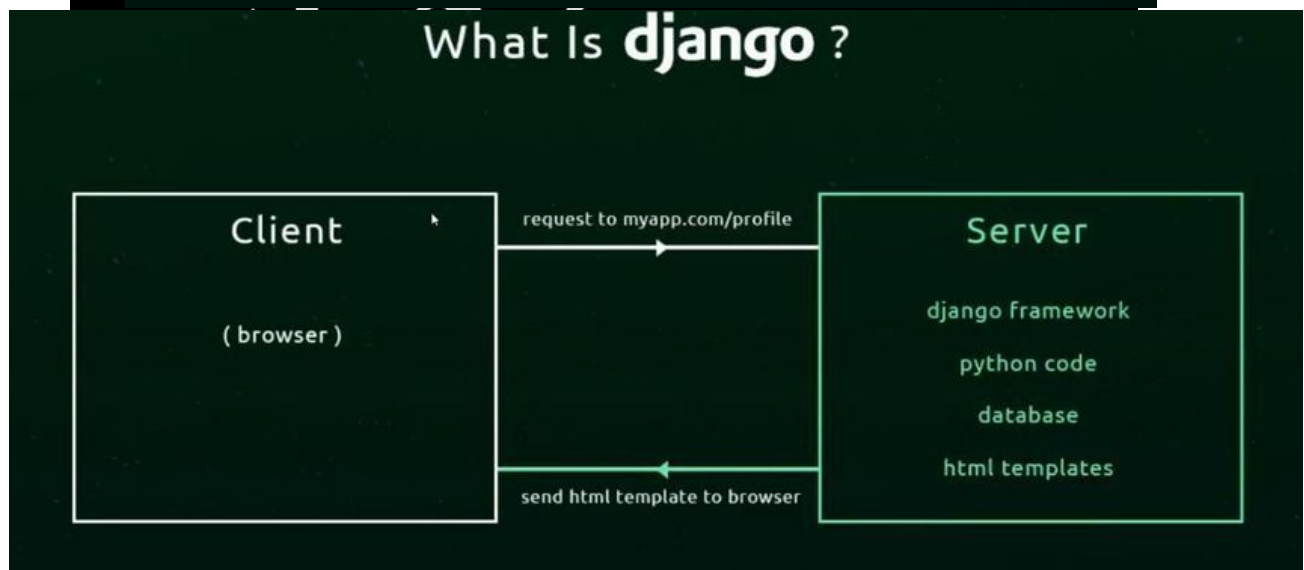
Software Required: Python3.9 / Online Editor

Theo



What Is django ?

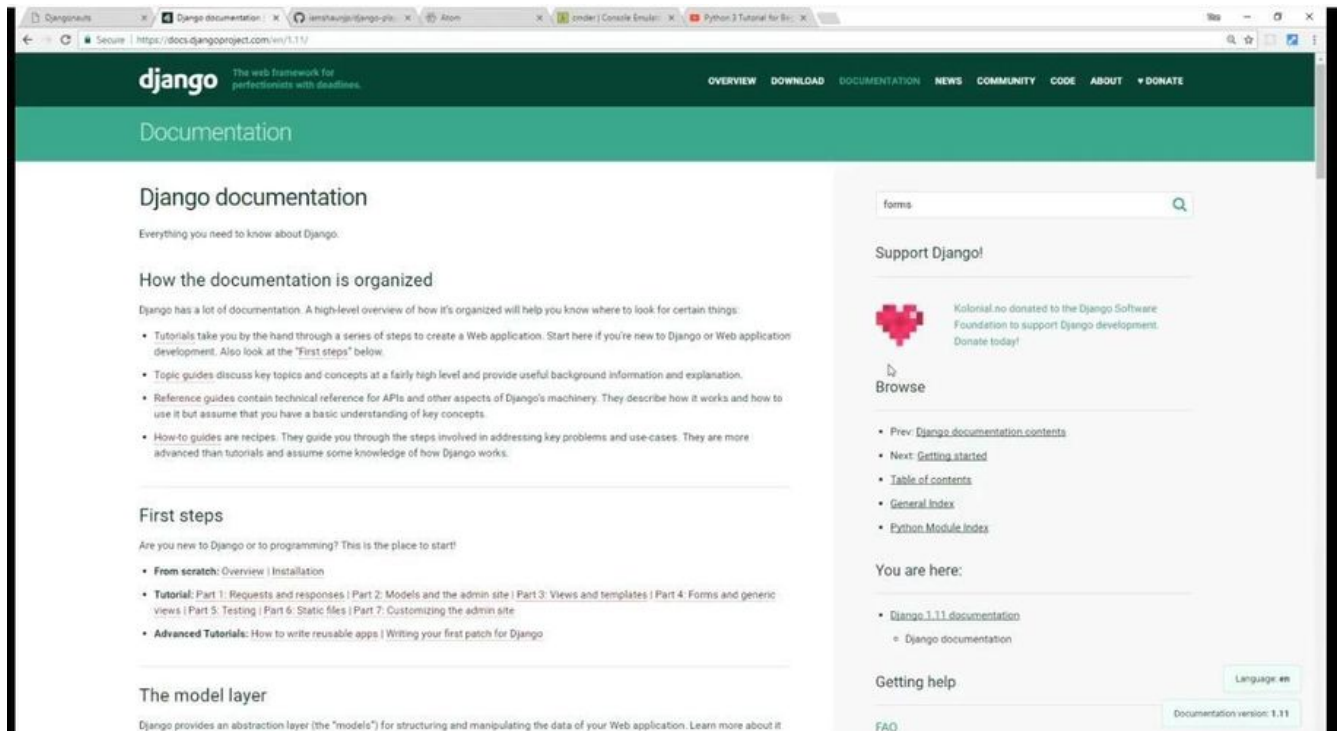
- A web application framework built with Python
- Comes baked with many cool features to help with...
 - User authentication
 - Templating language
 - Routing
 - ...And much more
- Allows us to easily create dynamic web apps using Python



Prepared by: Prof. Anand A. Ingle

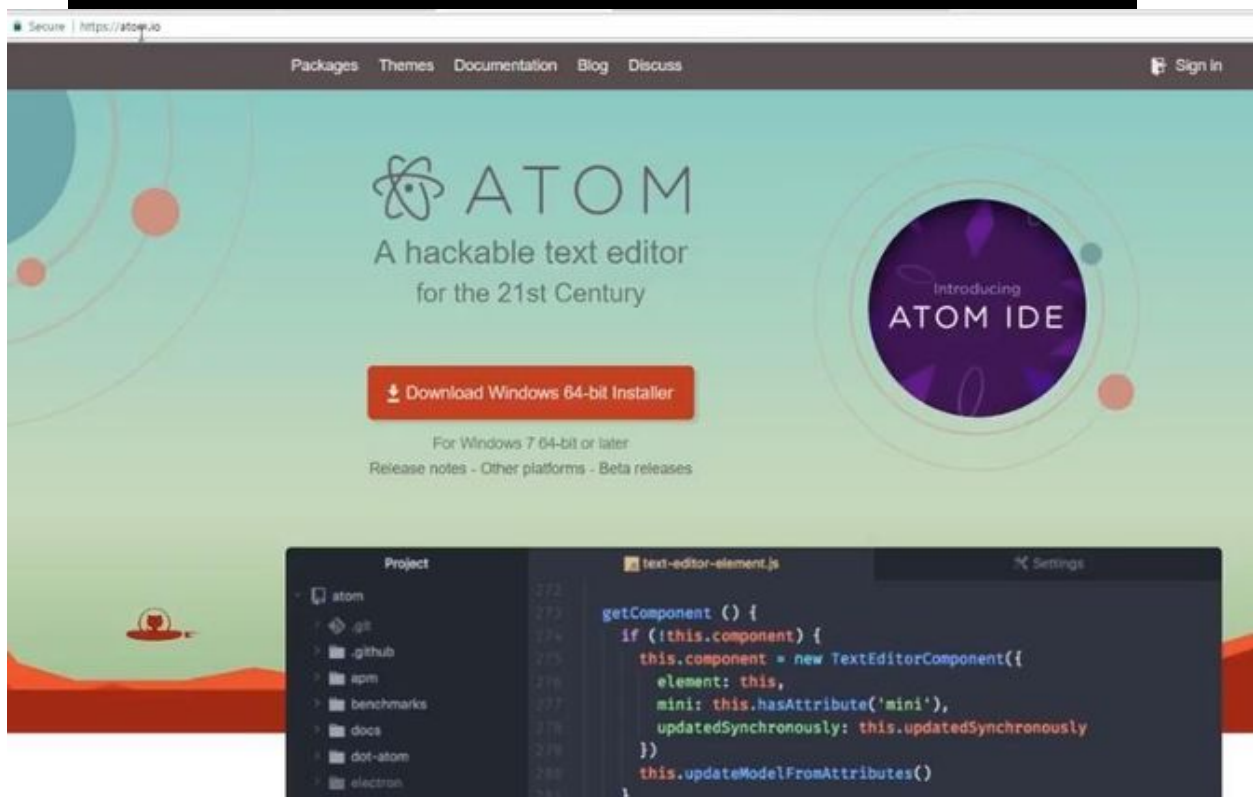
Django documentation is available at

<https://docs.djangoproject.com/en/4.0/>

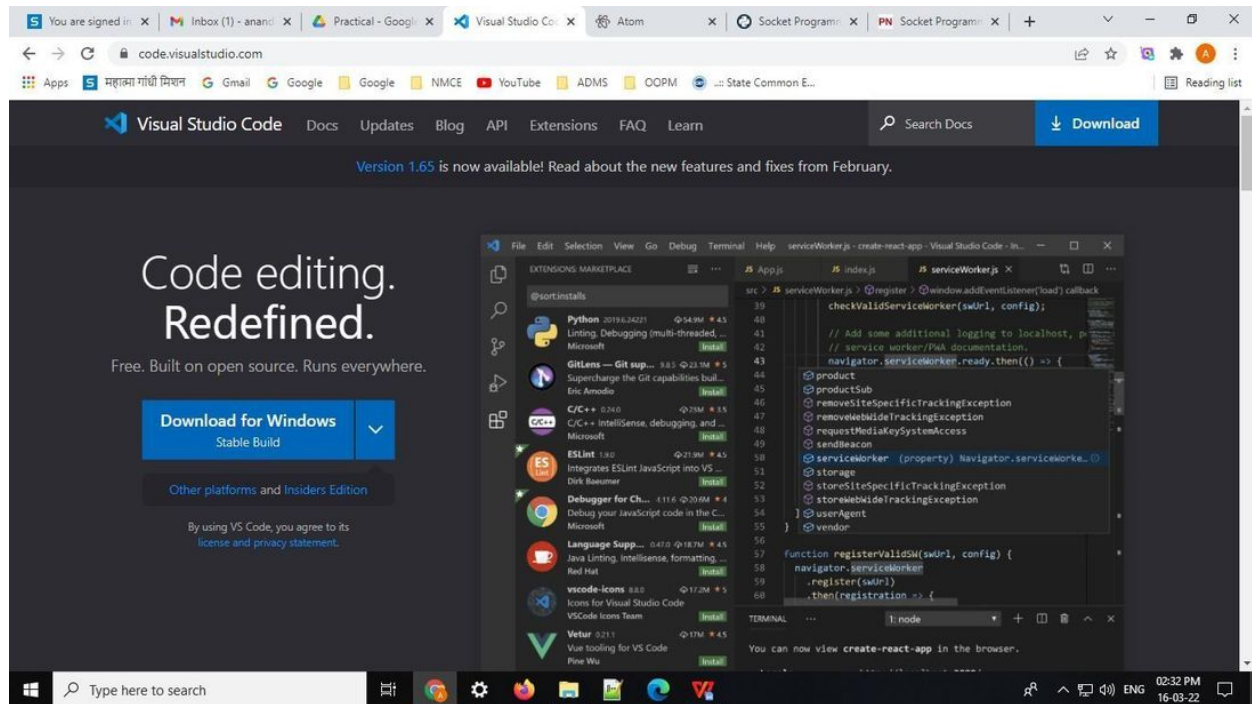


Download

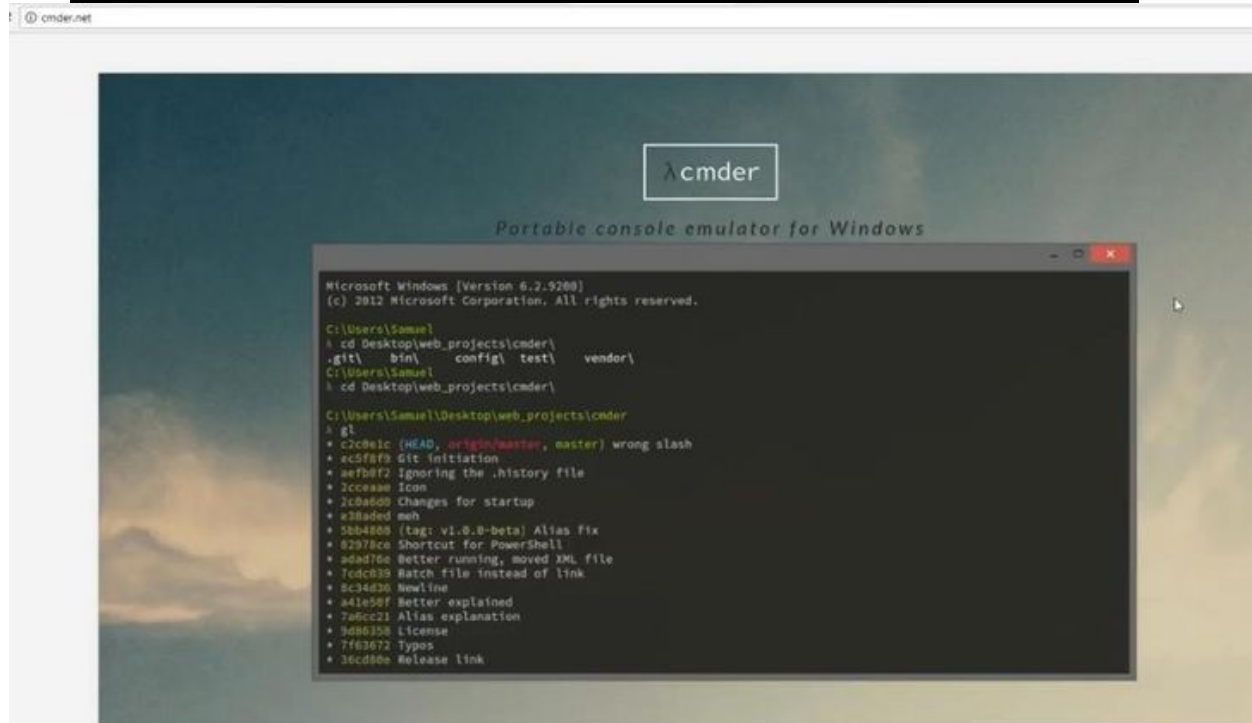
You can



Or you can download Visual Studio Code from <https://code.visualstudio.com/>



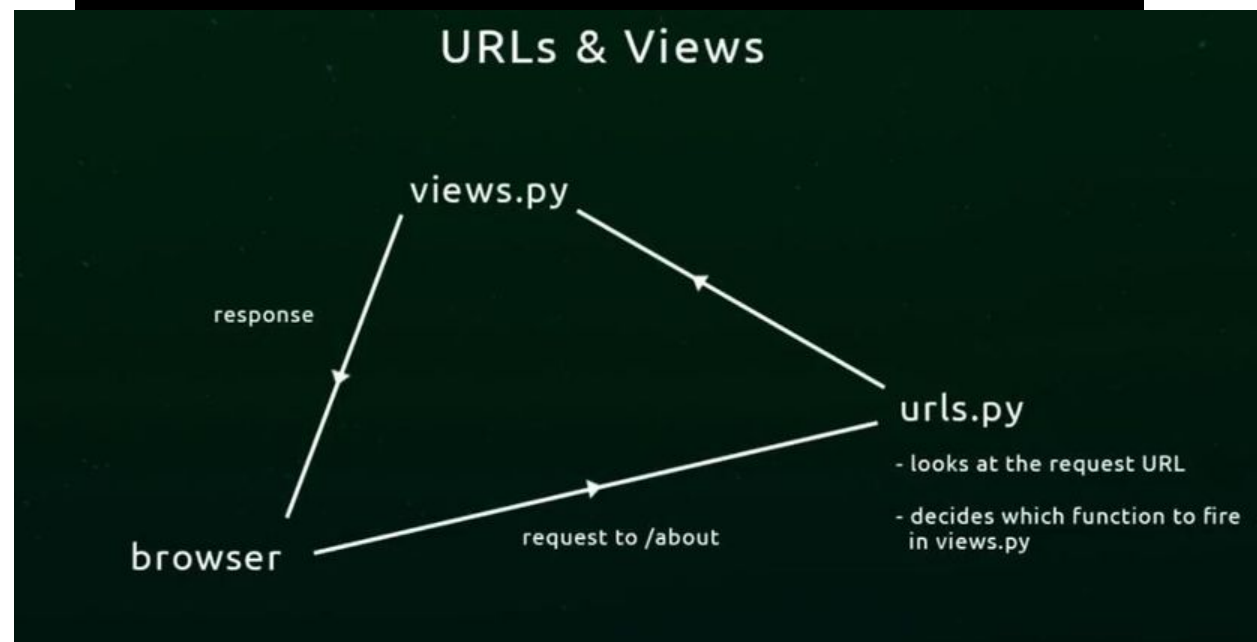
We d



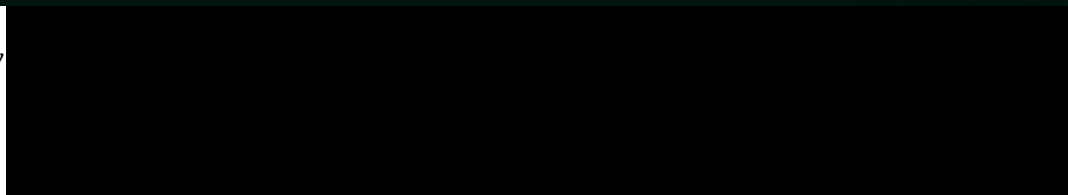
Cmder is a software package created out of pure frustration over the absence of nice console emulators on Windows. It is based on amazing software, and spiced up with the Monokai color scheme and a custom

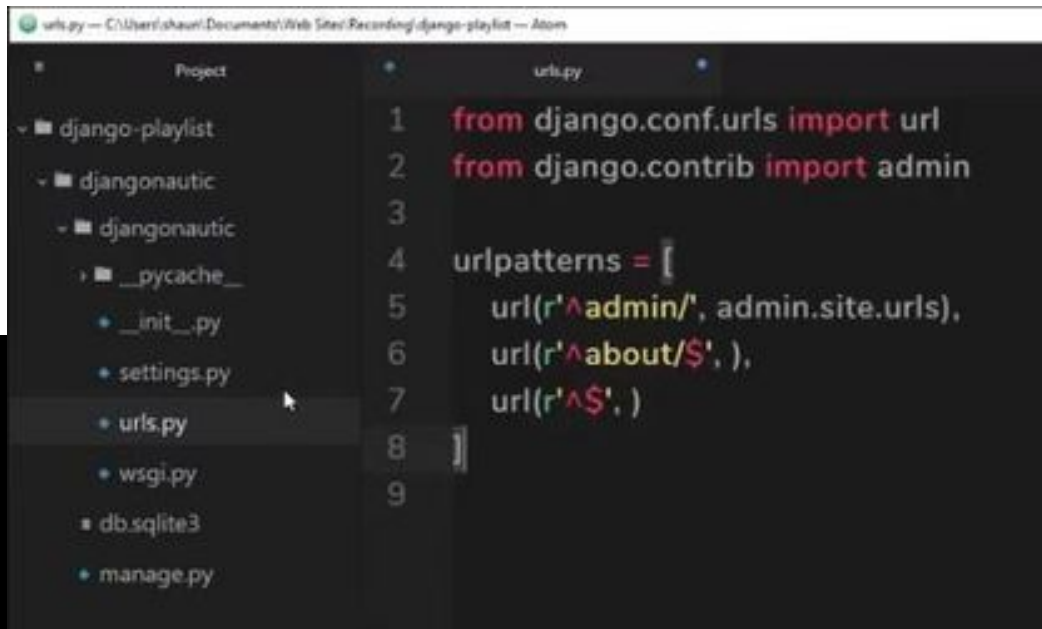
Now follow the steps:

- Install python
- Then install django using pip
 - pip install django
- Create a folder (directory) , where we want to create a django project
- Move to that folder in command prompt



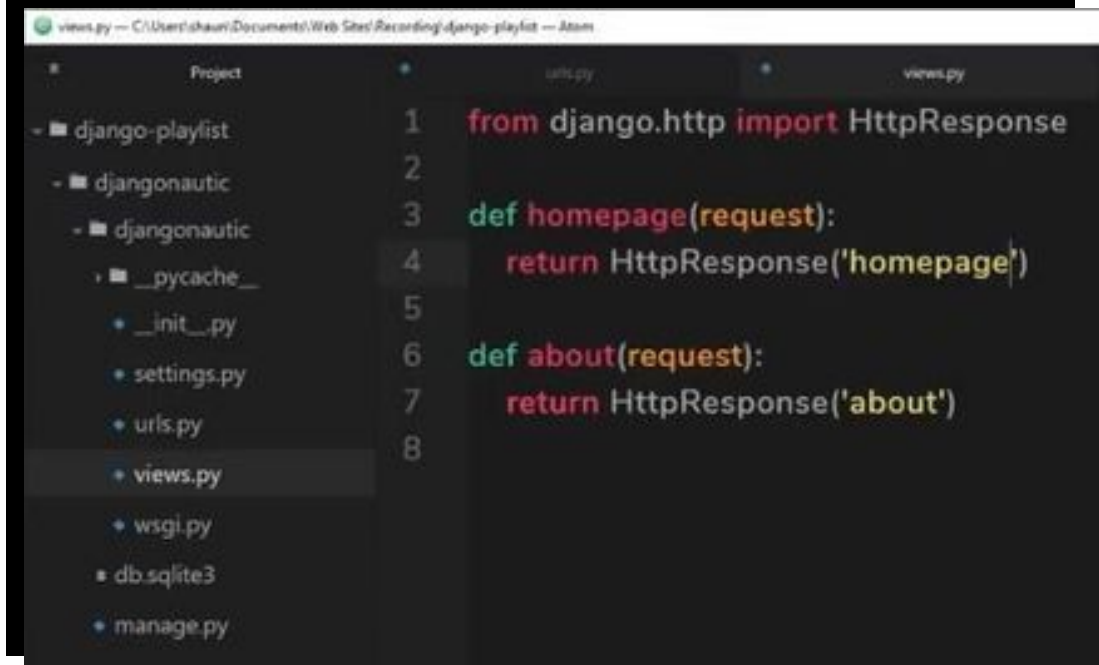
Now





```
1 from django.conf.urls import url
2 from django.contrib import admin
3
4 urlpatterns = [
5     url(r'^admin/', admin.site.urls),
6     url(r'^about/$', ),
7     url(r'^$', )
8 ]
9
```

Now we will create a views.py file in the same directory and
name it views.py. Our views.py file will look like this

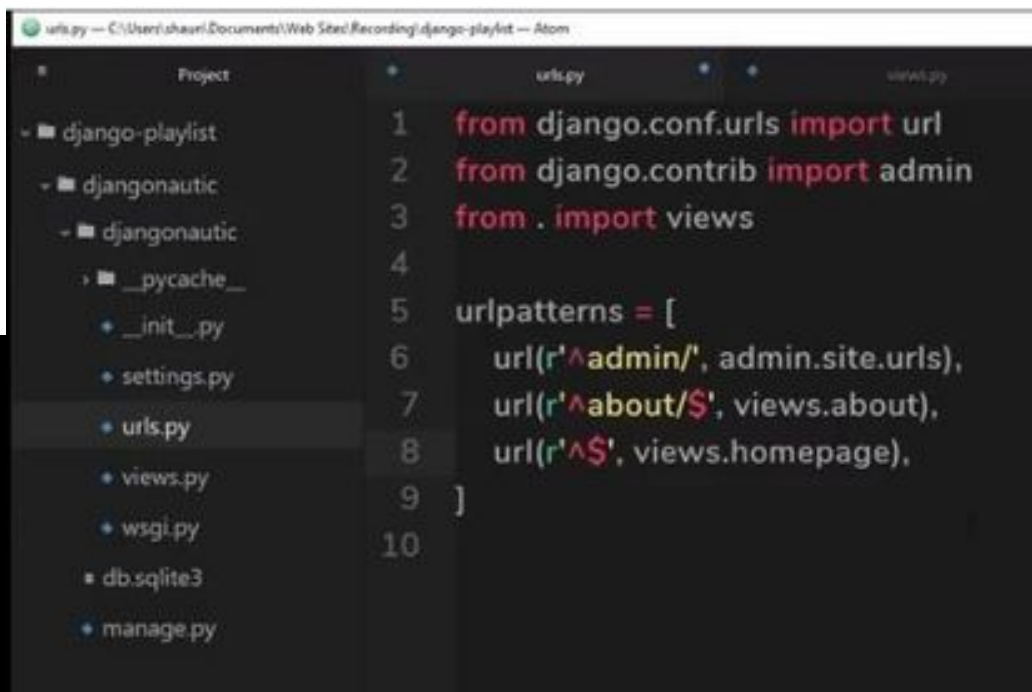


```
1 from django.http import HttpResponse
2
3 def homepage(request):
4     return HttpResponse('homepage')
5
6 def about(request):
7     return HttpResponse('about')
8
```

Then import views file in urls.py as below

→ `from . import views`

Our urls.py file will look like this



```
1 from django.conf.urls import url
2 from django.contrib import admin
3 from . import views
4
5 urlpatterns = [
6     url(r'^admin/', admin.site.urls),
7     url(r'^about/$', views.about),
8     url(r'^$', views.homepage),
9 ]
10
```

Now

→py

After
prom

→ [h](#)
chec

Now

➤
➤

C

command

3000 and

and for

Project -- C:\Users\shauri\Documents\Web Sites\Recording\django-playlist -- Atom

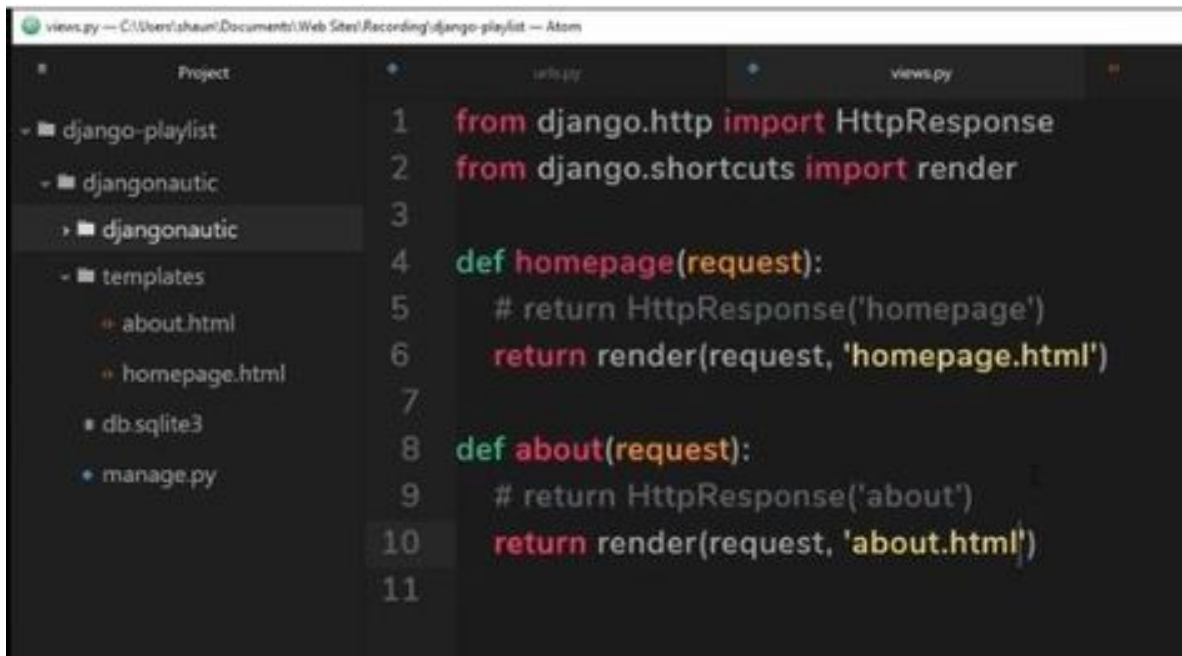
```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <meta charset="utf-8">
5     <title>Homepage</title>
6   </head>
7   <body>
8     <h1>This is the homepage</h1>
9     <p>Welcome to Djangonautic</p>
10  </body>
11 </html>
12
```

N

about.html -- C:\Users\shauri\Documents\Web Sites\Recording\django-playlist -- Atom

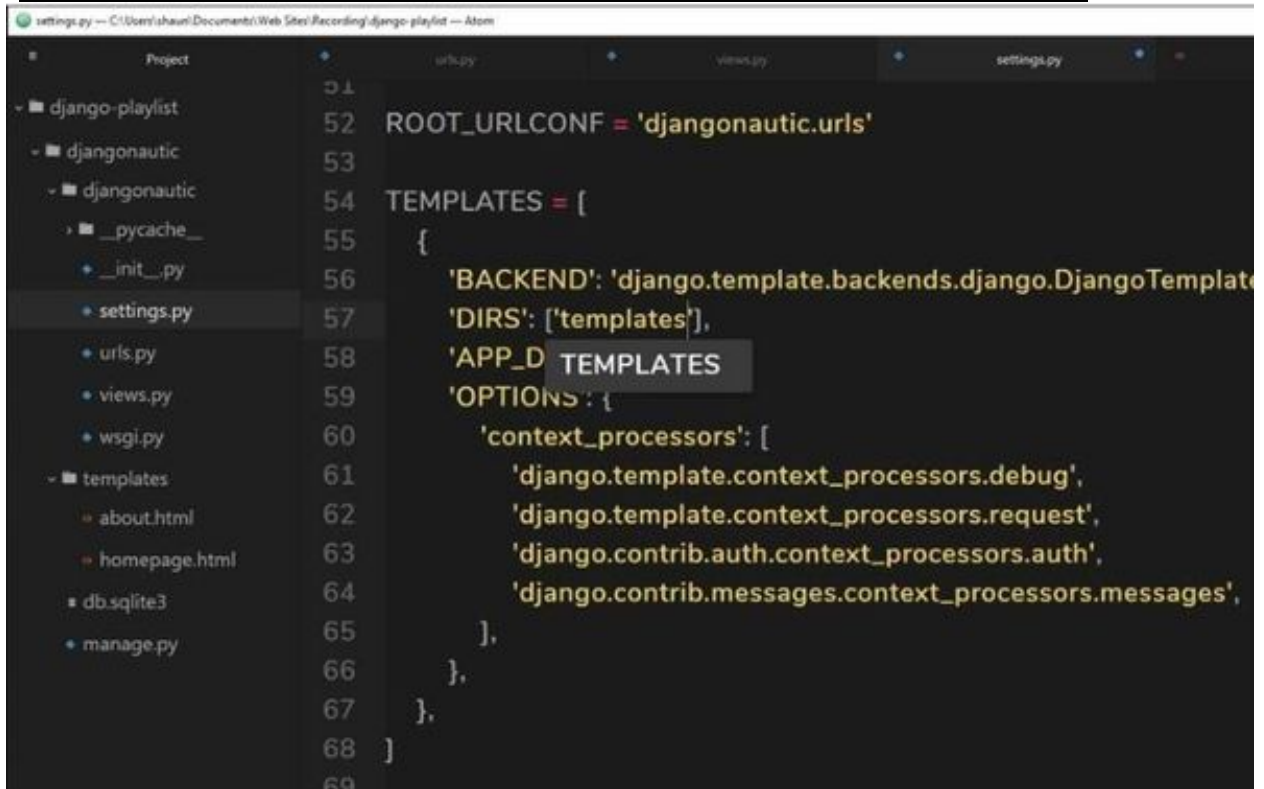
```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <meta charset="utf-8">
5     <title>About</title>
6   </head>
7   <body>
8     <h1>About us</h1>
9     <p>We like django.</p>
10  </body>
11 </html>
12
```

N

A screenshot of the Atom text editor showing the 'views.py' file. The left sidebar displays a project tree for 'django-playlist' with subfolders 'djagonautic' and 'templates'. The 'views.py' file is open in the main editor, showing Python code for Django views. The code includes imports for 'HttpResponse' and 'render', and two view functions: 'homepage' and 'about'.

```
1 from django.http import HttpResponse
2 from django.shortcuts import render
3
4 def homepage(request):
5     # return HttpResponse('homepage')
6     return render(request, 'homepage.html')
7
8 def about(request):
9     # return HttpResponse('about')
10    return render(request, 'about.html')
11
```

Now we need to update the settings.py file to include the templates directory.

A screenshot of the Atom text editor showing the 'settings.py' file. The left sidebar shows the project tree with 'settings.py' selected. The main editor displays the 'ROOT_URLCONF' and 'TEMPLATES' settings. The 'TEMPLATES' setting is a list containing a dictionary with 'BACKEND', 'DIRS', 'APP_DIRS', and 'OPTIONS' keys.

```
52 ROOT_URLCONF = 'djagonautic.urls'
53
54 TEMPLATES = [
55     {
56         'BACKEND': 'django.template.backends.django.DjangoTemplateBackend',
57         'DIRS': ['templates'],
58         'APP_DIRS': True,
59         'OPTIONS': {
60             'context_processors': [
61                 'django.template.context_processors.debug',
62                 'django.template.context_processors.request',
63                 'django.contrib.auth.context_processors.auth',
64                 'django.contrib.messages.context_processors.messages',
65             ],
66         },
67     },
68 ]
```

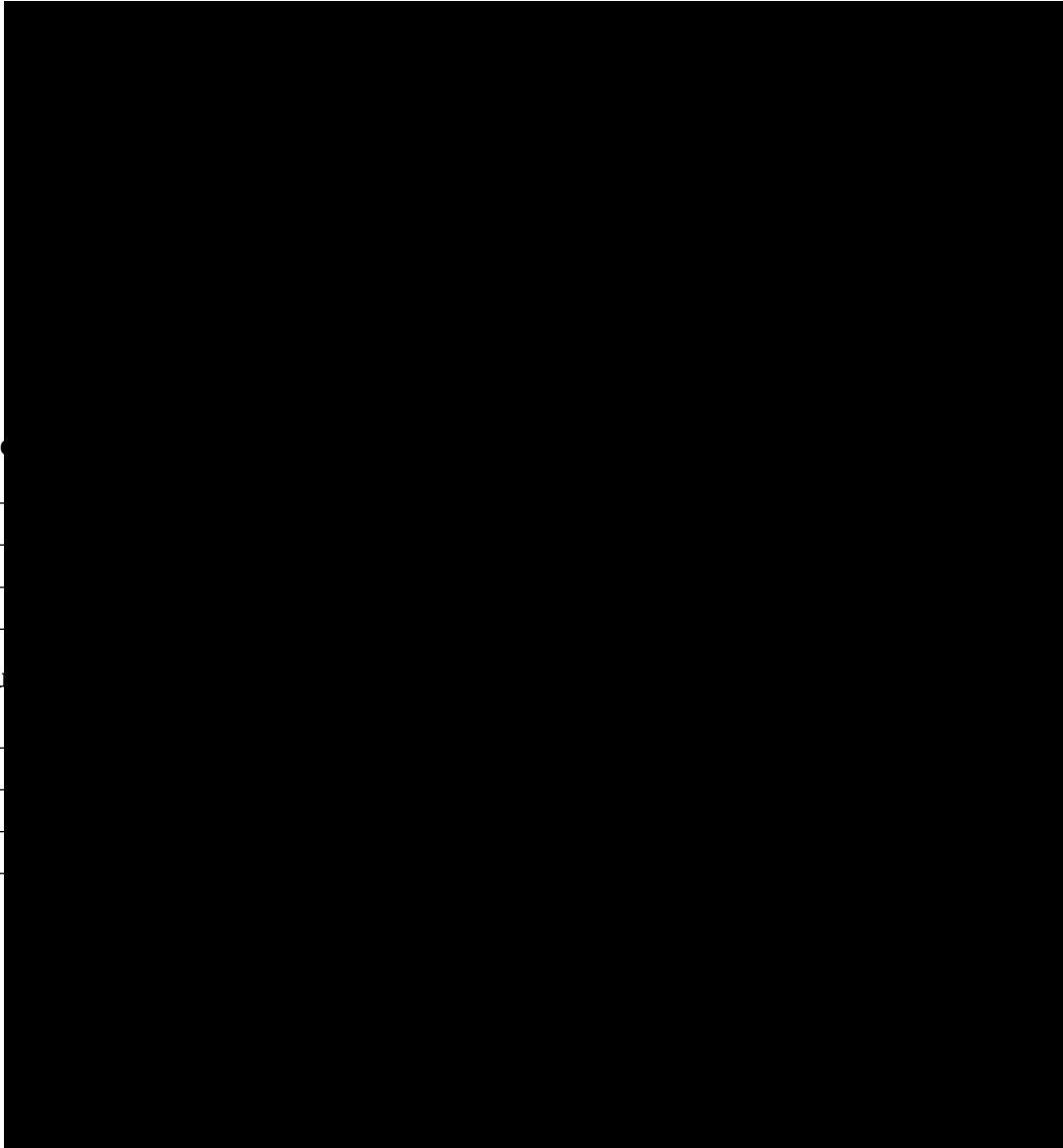
Now run our app now.

Steps for creating apps in Django

- Create app using the command as
→ `python manage.py startapp (app_name)`
- Create an `urls.py` file in that app folder, as we don't have it over there

Steps for using database in Django

- Create super user as



ange the

Con

Resu

Experiment No. 10

Name of Student			
Roll No.			
DOP	DOS	Marks/Grade	Sign

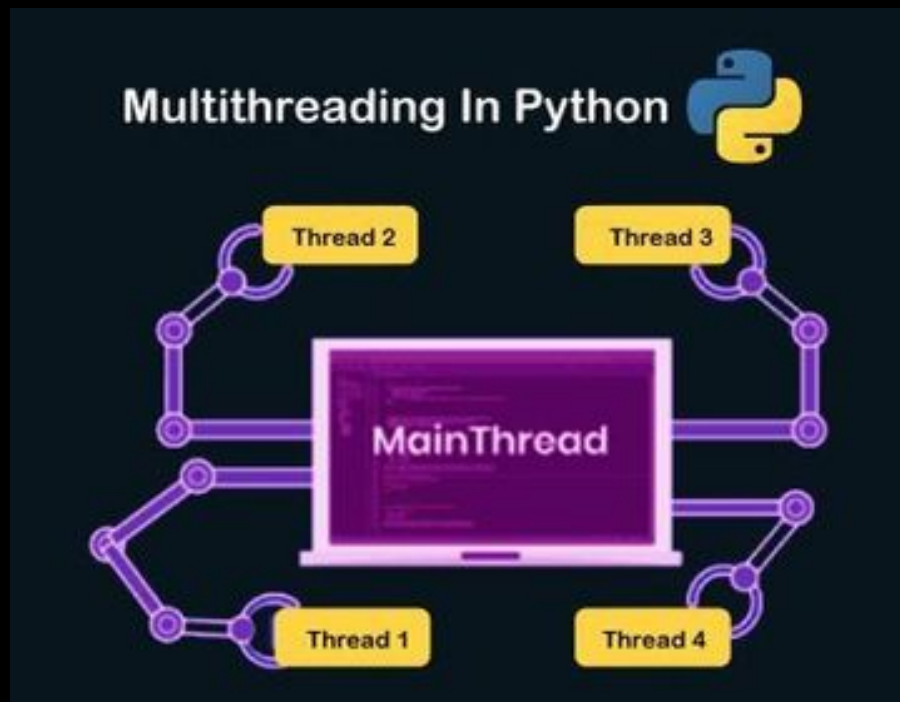
Aim: Programs on Threading using python.

Software Required: Python3.9 / Online Editor

Theory

A thread is a small unit of execution that can be created by the operating system. It is a part of a process and can execute independently. It is created by the operating system and can execute independently. It is created by the operating system and can execute independently.

It is created by the operating system and can execute independently. It is created by the operating system and can execute independently. It is created by the operating system and can execute independently.



Python

Multithreading is a threading technique in Python programming to run multiple threads concurrently by rapidly switching between threads with a CPU help (called context switching). Besides, it allows sharing of its data space with the main threads inside a process that share

Note: The Python Global Interpreter Lock (GIL) allows running a single thread at a time, even the machine has multiple processors.

Following are the benefits to create a multithreaded application in Python, as follows:

- 2
3
4
5
6

It is a common misconception that multitasking is a simple matter of running multiple tasks and simulating the user's interaction with the application. In reality, multitasking is a complex process that involves managing resources such as memory, CPU, and I/O. The user's interaction with the application is a critical component of multitasking, as it determines the length or block of time that the application spends on each task.

2

Synt

To implement the `thread` module in Python, we need to import a **thread** module and then define a function that performs some action by setting the target with a variable.

Threading Modules

The threading module is a high-level implementation of multithreading used to deploy an application in Python

To use multithreading, we need to import the threading module in Python Program.

Thread Class Methods

Method	Description
start()	Starts the thread. Only once for each thread.
run()	Runs the thread. Can be overridden by a class.
join()	Waits for the thread to finish.

Following are the steps to create a thread:

1. Import

Create

Syntax

import

A thread is a separate path of execution within a Python program.

2. Define *target function, arguments, and keyword arguments.*

- **Target:** It defines the function name that is executed by the thread.
- **Args:** It defines the arguments that are passed to the target function name.

For example:

```
import threading
```

```
def print_hello(n):
```



```
print("Hello, how old are you ", n)
```

```
t1 = threading.Thread( target = print_hello, args =(18, ))
```

In the above code, we invoked the **print_hello()** function as the target parameter. The **print_hello()** contains one parameter **n**, which passed to the **args** parameter.

3. Start a new thread: To start a thread in Python multithreading, call the thread class's object. The **start()** method can be called once for each thread object; otherwise, it throws an exception error.

Synt

t1.st

t2.st

4. Jo execution
and v completed, it
start

Join

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.

Outp

Hel

Tha

When the above program is executed, the **join()** method halts the execution of the main thread and waits until the thread **t1** is completely executed. Once the **t1** is successfully executed, the main thread starts its execution.

Note: If we do not use the **join()** method, the interpreter can execute any print statement inside the Python program. Generally, it executes the first print statement because the interpreter executes the lines of codes from the program's start.

5. Synchronizing Threads in Python

It is a thread synchronization mechanism that ensures no two threads can simultaneously execute a particular segment inside the program to access the shared resources. The situation may be termed as critical sections. We use a race condition to avoid the critical section condition, in which two threads do not access resources at the same time.

Conclusion:

Result

Experiment No. 11

Name of Student			
Roll No.			
DOP	DOS	Marks/Grade	Sign

Aim: Exploring basics of NumPy Methods.

Software Required: Python3.9 / Online Editor

Theory

What is NumPy?

NumPy is a library for

scientific computing with Python.

Why is it called NumPy?

Python is a high-level

language, and NumPy is a

library for scientific computing.

NumPy is a library for

scientific computing with Python.

Python

form

it ele

your

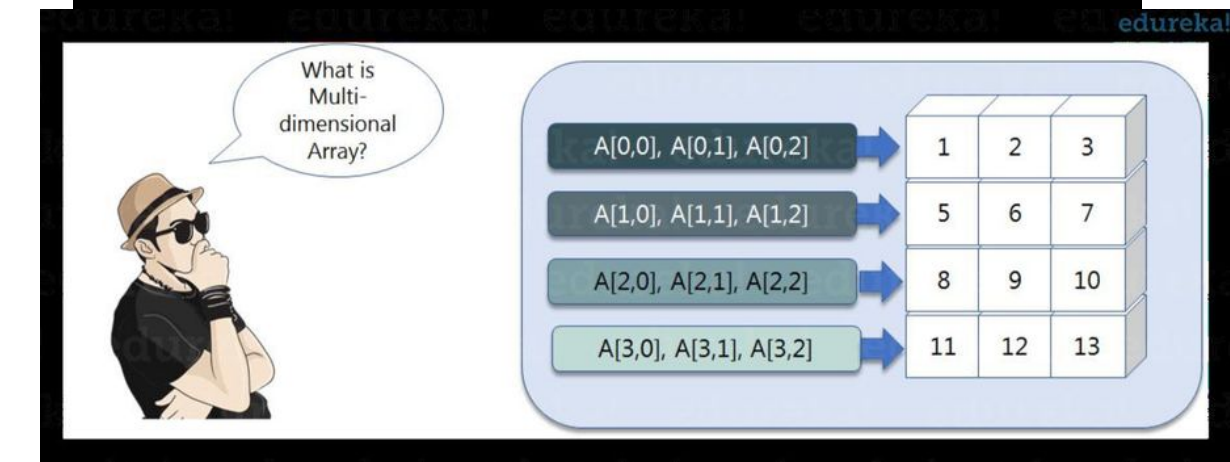
How

To in

insta

typin

let us



Here, I have different elements that are stored in their respective memory locations. It is said to be two dimensional because it has rows as well as columns. In the above image, we have 2

columns and 4 rows available.

How do I start NumPy?

Single-dimensional Numpy Array:

```
1import numpy as np
2a=np.array([1,2,3])
3print(a)
```

Output – [1 2 3]

Multi-dimensional Array:

```
1a=np
2print
```

O/P

[

Why

We u

1

2

3

The v

to lis

work

compared

venient to

r list.

```
1imp
```

```
2
```

```
3imp
```

```
4imp
```

```
5S=
```

```
6prin
```

```
7
```

```
8D=
```

```
9prin
```

O/P

The above output shows that the memory allocated by list (denoted by S) is 14000 whereas the memory allocated by the NumPy array is just 4000. From this, you can conclude that there is a major difference between the two and this makes Python NumPy array as the preferred choice over list.

Next, let's see how python NumPy array is faster and more convenient when compared to list.

```
1import time
```

```
2import sys
```

```
3
```

```

4SIZE = 1000000
5
6L1= range(SIZE)
7L2= range(SIZE)
8A1= np.arange(SIZE)
9A2=np.arange(SIZE)

```

```

10
11st
12re
13pr
14
15st
16re
17pr

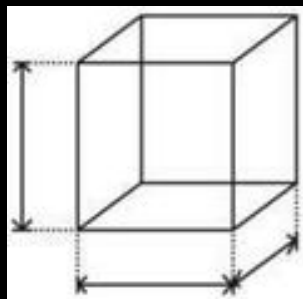
```

O/P :

In the
time
the d
num
had
num
with

There

Pyth



```

1imp

```

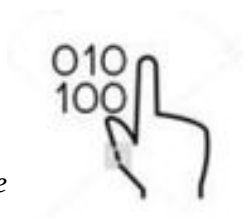
```

2a = np.array([1(1,2,3),(4,5,6)])
3print(a.ndim)

```

Output –

Since the output is _____ it is a two-dimensional array (multi dimension).



Prepare

Ingle

➤ **itemsizes:**

You can calculate the byte size of each element. In the below

code, I have defined a single dimensional array and with the help of 'itemsize' function, we can find the size of each element.

```
1import numpy as np
2a = np.array([(1,2,3)])
3print(a.itemsize)
```

Output –

So every element occupies _____ byte in the above numpy array.

• _____ want to
_____ will print
_____ e I have

```
1import
2a =
3print
```

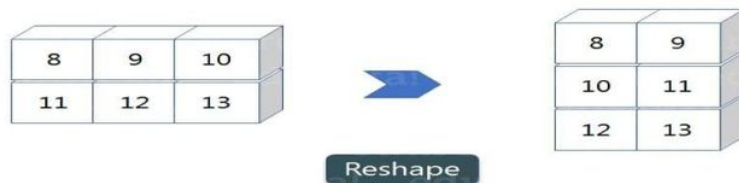
Output

As you can see in the above image, we have 3 columns and 2 rows which has converted into 2 columns and 3 rows. Let me show you practically how it's done.

```
1import
2a =
3print
4print
Output
```

Next, we will use the 'reshape' function to convert the array into a 2D array. The 'reshape' function is used to convert the array into a 2D array. The 'reshape' function is used to convert the array into a 2D array.

reshape
Reshape the array into a 2D array. The 'reshape' function is used to convert the array into a 2D array. The 'reshape' function is used to convert the array into a 2D array.



As you can see in the above image, we have 3 columns and 2 rows which has converted into 2 columns and 3 rows. Let me show you practically how it's done.

```

1import numpy as np
2a = np.array([(8,9,10),(11,12,13)])
3print(a)
4a=a.reshape(3,2)
5print(a)

```

Output – `[[ 8 9] [11 12] [12 13]]`

slicing

As you

Slicing

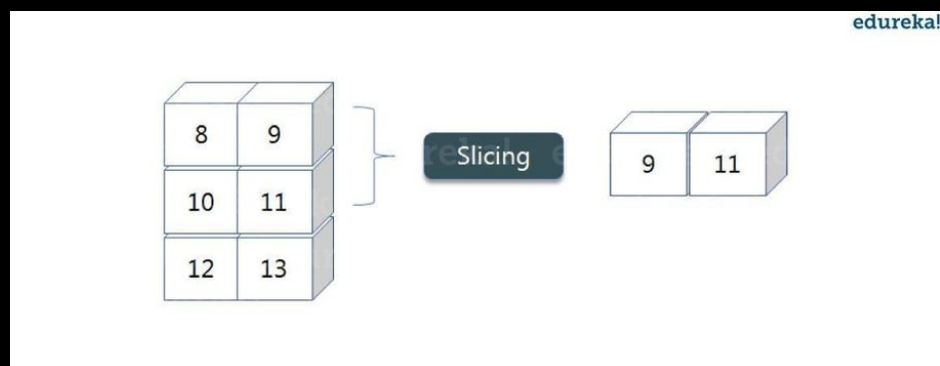
operation

follows

operation i.e

is slicing

consider the



by and we

example:

```

1import

```

```

2a=r

```

```

3prin

```

Here

There

Taking

array

by array.

index of the

```

1import

```

```

2a=np.array([(1,2,3,4),(3,4,5,6)])

```

```

3print(a[0:,2])

```

Output – `[ 3 5]`

Here colon represents all the rows, including zero. Now to get the 2nd element, we'll call index 2 from both of the rows which gives us the value 3 and 5 respectively.

Next, just to remove the confusion, let's say we have one more row and we don't want to get its 2nd element printed just as the image above. What we can do in such case?
Consider the below code:

```
1:import numpy as np
2:a=np.array([(8,9),(10,11),(12,13)])
3:print(a[0:2,1])
```

Output – []

As you can see, we have specified 0:2, this means that we want to get the first two rows. And 11 gets printed as the second element of the first row.

linspace
This function is used to generate a sequence of numbers. It is specified by the number of points and the range.

```
1:import numpy as np
2:a=np.linspace(0,10,5)
3:print(a)
```

Output – [0. 2. 4. 6. 8.]

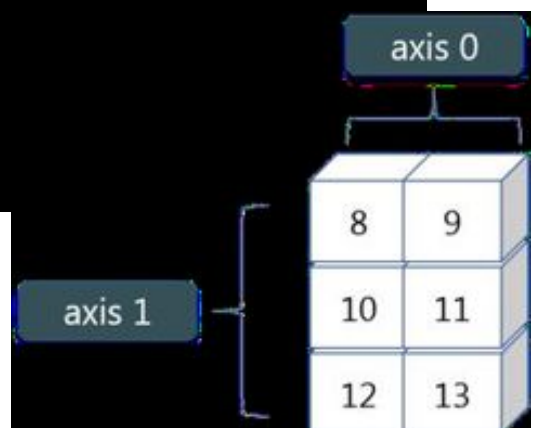
max
Next, we will see how to find the maximum value in an array. We can specify the axis as well. For example, we can find the maximum value along the first axis.

```
1:import numpy as np
2:a=np.array([(8,9),(10,11),(12,13)])
3:a.max()
4:print(a)
5:print(a.max())
6:print(a.max(axis=0))
7:print(a.max(axis=1))
```

Output – [8 9 10 11 12 13]

You can see that the output is a 1D array. This is because we have specified the axis. If we don't specify the axis, it will return the maximum value of the entire array.

As you can see in the figure, we have a numpy array 2*3. Here the rows are called as axis 1 and the columns are called as axis 0. Now you must be wondering what is the use of these axis?



Suppose you want to calculate the sum of all the columns, then you can make use of axis. Let us see practically, how you can implement axis in your PyCharm:


```
1a= np.array([(1,2,3),(3,4,5)])
2print(a.sum(axis=0))
Output – [      ]
```

Therefore, the sum of all the columns are added where $1+3=4$, $2+4=6$ and $3+5=8$. Similarly, if you replace the axis by 1, then it will print [6 12] where all the rows get added.

Square

There are many ways to find the square of a number. You can find the square of a number in the following ways:

```
1import numpy as np
2a= np.array([(1,2,3),(3,4,5)])
3print(a**2)
4print(a*a)
```

As you can see, the output is the same. Also, the standard deviation of the array is the same from the mean.

Add

You can add two arrays using the `+` operator and division using the `/` operator.

```
1import numpy as np
2x= np.array([(1,2,3),(3,4,5)])
3y= np.array([(1,2,3),(3,4,5)])
4print(x+y)
Output – [      ]
```

This is the same as the subtraction, multiplication and division.

```
1import numpy as np
2x= np.array([(1,2,3),(3,4,5)])
3y= np.array([(1,2,3),(3,4,5)])
4print(x-y)
5print(x*y)
6print(x/y)
```

```
Output – [[          ] [          ]]
[[          ] [          ]]
[[          ] [          ]]
```

Vertical & Horizontal Stacking

Next, if you want to concatenate two arrays and not just add them, you can perform it using two ways – vertical stacking and horizontal stacking.

```
1import numpy as np
```

```
2x= np
```

```
3y= np
```

```
4print
```

```
5print
```

Output

```
[[
```

rave

There

i.e row

e column

```
1imp
```

```
2x= np
```

```
3print
```

Output

There

let's

modu

etc. First,

import a

```
1imp
```

```
2imp
```

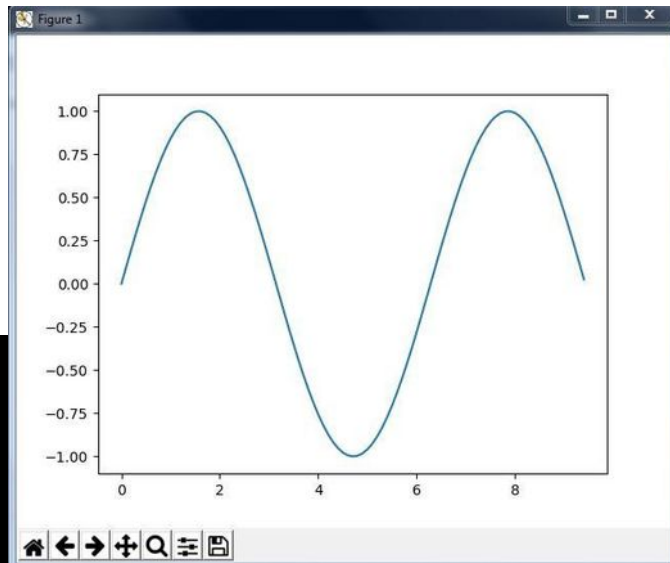
```
3x= np
```

```
4y= np
```

```
5plt.
```

```
6plt.
```

Output

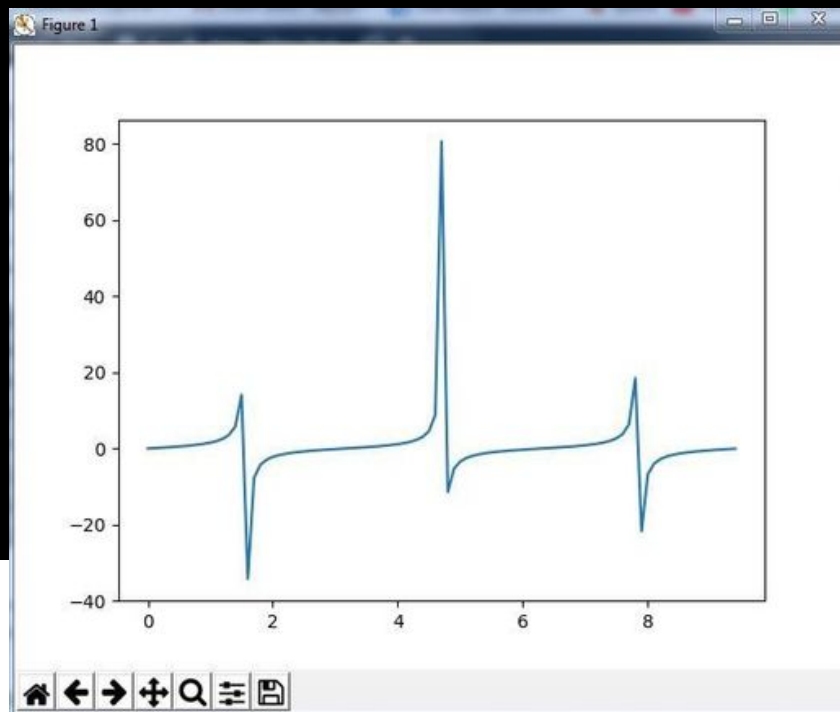


Similar

say $\tan$.

1:imp
2:imp
3x=
4y=n
5plt.
6plt.

Outp



let's see some other special functionality in numpy array such as exponential and logarithmic function. Now in exponential, the e value is somewhere equal to 2.7 and in log, it is actually log

base 10. When we talk about natural log i.e log base e, it is referred as Ln. So let's see how it is implemented practically:

```
1a= np.array([1,2,3])  
2print(np.exp(a))
```

Output – [

As you can see the above output, the exponential values are printed i.e e raise to the power 1 is e, which gives the result as 2.718... Similarly, e raise to the power of 2 gives the value some implement it:

```
1imp  
2imp  
3a=   
4prin
```

Outp

Here if we want log b

```
1imp  
2imp  
3a=   
4prin
```

Outp

Conc

Resu

Experiment No. 12

Name of Student			
Roll No.			
DOP	DOS	Marks/Grade	Sign

Aim: Program to send email and read content of URL.

Software Required: Python3.9 / Online Editor

The objective of this experiment is to write a Python program to send an email and read the content of a URL. Python, being a high-level programming language, has a rich library of modules and packages. The `smtplib` module is used to send emails. The `urllib` module is used to read the content of a URL. Here, we will use the `smtplib` module to send an email and the `urllib` module to read the content of a URL. If you want to send an email, you need to provide the following information: sender's email address, receiver's email address, and the content of the email.

Steps:

First, we will create an SMTP object. After that, we will connect to the SMTP server. In the next step, we will login to the SMTP server. In the fourth step, we will send an email. In the fifth step, we will read the content of a URL. In the sixth step, we will terminate the SMTP session.

Layer: The application layer of the OSI model is responsible for the communication between the application and the network. In this layer, you need to provide the email address and the content of the email. The application layer also handles the authentication of the user.

Installation: The `smtplib` module is installed by default in Python. You can check the installation by using the following command: `python -c 'import smtplib'`. If you receive an error, you need to install the `smtplib` module. For our task, we will use the `smtplib` module to send an email.

Code: The following code illustrates how to send an email and read the content of a URL. The code uses the `smtplib` module to send an email and the `urllib` module to read the content of a URL. The code also uses the `sys` module to handle the command-line arguments.

Python code to illustrate Sending mail from
your Gmail account

```
import smtplib
```

```
# creates SMTP session
```

```
s = smtplib.SMTP('smtp.gmail.com', 587)
```

```
# start TLS for security
```

```
s.starttls()
```

```
# Authentication
```

```
s.login("sender_email_id", "sender_email_id_password")
```

```
# message to be sent
```

```
mess
```

```
# ser
```

```
s.send
```

```
# ter
```

```
s.quit
```

```
Send
```

```
For
```

```
inser
```

```
initia
```

```
# Py
```

```
# to
```

```
# fro
```

```
impo
```

```
# list
```

```
li =
```

```
for d
```

```
s =
```

```
s.s
```

```
s.l
```

```
message = "Message_you_need_to_send"
```

```
s.sendmail("sender_email_id", dest, message)
```

```
s.quit()
```

Important Points:

Prepared by: Prof. Anand A. Ingle

- *This code can send simple mail which doesn't have any attachment or any subject.*
- *One of the most amazing things about this code is that we can send any number of emails using this and Gmail mostly put your mail in the primary section. Sent mails would not be detected as Spam generally.*
- *File handling can also be used to fetch email id from a file and further used for sending the emails.*

Conclusion:

Result:

References

1. [http://www.geogebra.org/m/...](#)
2. [http://www.geogebra.org/m/...](#)
3. [http://www.geogebra.org/m/...](#)
4. [http://www.geogebra.org/m/...](#)
5. [http://www.geogebra.org/m/...](#)
6. [http://www.geogebra.org/m/...](#)
7. [http://www.geogebra.org/m/...](#)

